



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Надежда Јевремовић

Евалуација различитих софтверских имплементација машина стања на ресурсно ограниченим системима

МАСТЕР РАД
Мастер академске студије

Ментор: др Иван Мезеи

Нови Сад, 2026



КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:			
Идентификациони број, ИБР:			
Тип документације, ТД:	МОНОГРАФСКА ПУБЛИКАЦИЈА		
Тип записа, ТЗ:	ТЕКСТУАЛНИ ШТАМПАНИ МАТЕРИЈАЛ		
Врста рада, ВР:	МАСТЕР РАД		
Аутор, АУ:	Надежда Јевремовић		
Ментор, МН:	др Иван Мезеи, редовни професор		
Наслов рада, НР:	Евалуација различитих софтверских имплементација машина стања на ресурсно ограниченим системима		
Језик публикације, ЈП:	српски		
Језик извода, ЈИ:	српски		
Земља публикавања, ЗП:	Република Србија		
Уже географско подручје, УГП:	Војводина		
Година, ГО:	2026.		
Издавач, ИЗ:	АУТОРСКИ РЕПРИНТ		
Место и адреса, МА:	Нови Сад, Трг Доситеја Обрадовића 6		
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)			
Научна област, НО:	Електротехника и рачунарство		
Научна дисциплина, НД:	Ембедед системи		
Предметна одредница/Кључне речи, ПО:			
УДК			
Чува се, ЧУ:	Библиотека ФТН, Нови Сад, Трг Доситеја Обрадовића 6		
Важна напомена, ВН:	НЕМА		
Извод, ИЗ:			
Датум прихватања теме, ДП:			
Датум одбране, ДО:			
Чланови комисије, КО:	Председник:	др Предраг Теодоровић, ванредни професор	
	Члан:	др Богдан Павковић, ванредни професор	Потпис ментора
	Члан, ментор	Др Иван Мезеи, редовни професор	



KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :		Monographic publication	
Type of record, T3 :		Textual material, printed	
Contents code, CC :		Master's thesis	
Author, AU :		Nadežda Jevremović	
Mentor, MN :		Ivan Mezei, PhD	
Title, TI :		Evaluation of different software state machine implementations on resource-constrained systems	
Language of text:, LT :		Serbian	
Language of abstract, LA :		Serbian	
Country of publication, CP :		Serbia	
Locality of publication, LP :		Vojvodina	
Publication year, PY :		2026	
Publisher, PB :		Author's reprint	
Publication place, PP :		Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6	
Physical description, PD : (chapters/ pages/ ref. / tables/ pictures/ graphs/ appendixes)			
Scientific field, SF :		Electrical engineering	
Scientific discipline, SD :		Embedded systems and algorithms	
Subject/ Key words, S/KW :			
UC			
Holding data, HD :		Library of the Faculty of Technical Sciences 21000 Novi Sad, Trg Dositeja Obradovića 6	
Note, N :			
Abstract, AB :		.	
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended board, DB :	President:	Predrag Teodorović, PhD, associate professor	
	Member:	Bogdan Pavković, associate professor	Mentor's signature
	Member, Mentor	Ivan Mezei, PhD, full professor	



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

Број:

ЗАДАТАК ЗА ЗАВРШНИ РАД

Датум: 15.9.2025.

(Податке уноси предметни наставник - ментор)

Студијски програм:	Енергетика, електроника и телекомуникације		
Студент:	Надежда Јевремовић	Број индекса:	E1 92/2024
Степен и врста студија:	Мастер академске студије		
Област:	Ембедед системи и алгоритми		
Ментор:	др Иван Мезеи		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:			
- проблем – тема рада;			
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Евалуација различитих софтверских имплементација машина стања на ресурсно ограниченим системима

ТЕКСТ ЗАДАТКА:

--

Руководилац студијског програма:	Ментор рада:
др Јован Бајић	др Иван Мезеи

Примерак за: ☐ - Студента; ☐ - Ментора



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана 15.9.2026.

Ментор

**Изјава о академској
честитости**

Студент: **Надежда Јевремовић**

Број индекса: **E1 92/2024**

Студенткиња мастер академских студија

Ауторка рада под називом: **Евалуација различитих софтверских имплементација
машина стања на ресурсно ограниченим системима**

Потписивањем изјављујем:

- да је рад искључиво резултат мог сопственог истраживачког рада;
- да нисам користио алате вештачке интелигенције за генерисање и/или креирање делова рада;
- да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао и наведени цу у списку литературе/референци који су саставни део овог рада;
- да сам добио све дозволе за коришћење ауторског дела који се у целости уносе у предати рад и да сам то јасно навео;
- да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторском и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Новом Саду;
- да сам свестан да плагијат укључује и представљање, употребу и дистрибуирање рада предавача или других студената као сопствених;
- да сам свестан последица које код доказаног плагијата могу проузроковати на предати рад и мој статус;
- да је електронска верзија рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета.

У Новом Саду, **15.9.2026.**

Потпис студента/студенткиње

Садржај

1.	Увод.....	9
2.	Теоријске основе.....	11
2.1	Угнеждена <i>Switch-case FSM</i>	11
2.2	Модификована угнеждена <i>Switch-case FSM</i>	13
2.3	<i>FSM</i> структурног низа	14
2.4	Директни <i>LUT FSM</i>	16
2.5	<i>FSM</i> објеката стања.....	18
2.6	Планарни <i>HSM FSM</i>	20
2.7	Угнеждени <i>HSM FSM</i>	22
2.8	Индиректни <i>LUT FSM</i> (<i>Santic</i>).....	24
3.	Методологија.....	26
3.1	Елементарна машина стања	26
3.2	Проширена машина стања	27
3.3	Комплексна машина стања	29
3.4	Хардверске платформе	30
3.5	Методологија мерења циклуса	30
3.5.1	Зашто није коришћен <i>ISR</i> приступ?.....	31
3.5.2	<i>Polling</i> приступ коришћен за мерење.....	32
3.5.3	Регистри <i>Timer1</i> коришћени за мерење	32
3.5.4	Заједничка функција за мерење транзиције	34
3.6	Нивои оптимизације компајлера	34
4.	Резултати мерења и дискусија резултата.....	36
4.1	Резултати на елементарној машини стања	36
4.1.1	<i>AVR</i> платформа.....	36
4.1.2	Поређење <i>AVR</i> резултата са литературом	39
4.1.3	<i>ESP32</i> платформа	40
4.1.4	Упоредна анализа <i>AVR</i> и <i>ESP32</i>	44
4.1.5	Разлагање трошка индиректних приступа.....	45
4.2	Резултати на проширеној (<i>MQTT</i>) машини стања	47
4.2.1	<i>AVR</i> платформа.....	47
4.2.2	<i>ESP32</i> платформа	49

4.2.3 Упоредна анализа AVR и ESP32	53
4.3 Резултати на комплексној машини стања	54
4.3.1 AVR платформа.....	54
4.3.2 ESP32 платформа	54
4.3.3 Упоредна анализа AVR и ESP32	54
4.4 Поређење скалирања кроз три нивоа машине стања (елементарна → проширена → комплексна, кључни закључак целог рада)	54
4.5 Закључна дискусија резултата	54
Литература.....	55

1. Увод

Конечне машине стања, или краће машине стања (*eng. Finite State Machine, FSM*) представљају модел понашања система заснован на коначном скупу стања. Прелазак из једног стања у друго дешава се према унапред дефинисаним правилима, у зависности од догађаја који се јаве у систему, а ти догађаји су по правилу последица промена на улазу система или промене стања система. Ова техника нашла је примену у широком спектру области попут комуникационих протокола, системима управљања, графичким корисничким интерфејсима. Посебно је значајна у развоју софтвера за уграђене системе (*eng. embedded systems*) — рачунарске системе уграђене унутар неког уређаја, најчешће засноване на микроконтролерима, наменски пројектоване да обављају одређену, ограничену функцију уз строга ограничења у погледу расположиве меморије, процесорске снаге и потрошње енергије. На оваквим системима, јасно дефинисана и предвидива логика коју машина стања пружа, чиме олакшава пројектовање поузданог понашања упркос ограниченим ресурсима. Иако је концепт машине стања теоријски добро утемељен, начин на који се она преводи у изворни код програмског језика није јединствен. Исти проблем може бити решен на неколико суштински различитих начина, при чему избор између њих није увек очигледан, нити подједнако документован у литератури. Упркос томе, не постоји систематична анализа која би упоредила утицај различитих начина имплементације на перформансе и заузеће ресурса на платформама са ограниченим ресурсима, што уједно представља и главну мотивацију за овај рад.

Пројектовање машине стања укључује три корака: идентификацију свих могућих стања система, идентификацију свих догађаја који могу изазвати промену тог стања или извршавање неке акције, и дефинисање скупа акција које систем предузима за сваку комбинацију стања и догађаја. У програмском језику C, машина стања се најчешће представља преко угнеждених *switch* исказа, или преко табеле претраге (*eng. lookup table*). Табела претраге представља концизнији начин записа, али, за разлику од *switch-a*, нема уграђену заштиту за неважеће комбинације стања и догађаја, па се граница мора проверити ручно [JSnd].

Мотивацију за полазну тачку овог рада представља запажање да два наизглед различита начина записивања исте логике носе различите практичне последице, при чему ни поређење ова два конкретна приступа не даје мерљив, квантификован одговор о томе колико та разлика заправо кошта [JSnd]. Ако постоји мерљива разлика између само два поређена приступа, поставља се питање да ли постоји слична или израженија разлика између знатно већег броја имплементационих приступа доступних у пракси и у литератури. Ови приступи обухватају угнеждени *switch* у различитим стилским варијантама, структурни низ, директне и индиректне табеле претраге, низове функција по стању и хијерархијску организацију стања. Упркос широкој доступности ових приступа, не постоји систематична анализа која би упоредила њихов утицај на перформансе и заузеће ресурса на платформама са ограниченим ресурсима — управо овај недостатак представља централни истраживачки проблем којим се овај рад бави.

То питање додатно је поткрепљено и следећим запажањем: уочено је да резултати заузећа меморије и времена извршавања, приказани у раду који пореди шест различитих

архитектонских приступа имплементација машина стања у програмском језику C, показују да се рангирање приступа по меморијској ефикасности битно разликује у зависности од примењеног оптимизационог прекидача [CO23]. Ово указује на зависност архитектонског избора и нивоа оптимизације преводиоца, што поређење имплементационих приступа чини сложенијим од очекиваног.

Постојећа литература о начинима пројектовања машина стања углавном даје теоријску, квалитативну оцену сложености појединих приступа, без систематског, упоредивог емпиријског мерења на конкретном, ресурсно ограниченом хардверу [PA06], [CO23]. Овај рад полази управо од те чињенице. Како имплементациони приступи не би остали упоређени само на нивоу теорије, сваки од њих је имплементиран, измерен и упоређен на стварним микроконтролерским платформама, чиме се теоријске тврдње из литературе емпиријски проверавају, квантификују, и, где је то случај, допуњују или оспоравају. Предмет овог рада је поређење осам имплементационих приступа машина стања на различитим развојним платформама.

Сваки од ових приступа имплементиран је над три различита функционална аутомата различите сложености и измерен на две архитектонски различите платформе: *ATmega328P* (*Arduino Uno*, 8-битна *AVR* архитектура) и *ESP32* (32-битна *Xtensa* архитектура). За сваку имплементацију мерено је заузеће програмске и радне меморије, као и број процесорских циклуса по транзицији, на више нивоа оптимизације преводиоца. Добијени резултати упоређени су са теоријским тврдњама из релевантне литературе, чиме се утврђује у којој мери се теоријски очекивано понашање поклапа са стварно измереним. Резултати овог рада релевантни су у ширем контексту развоја софтвера за уграђене системе, чији значај континуирано расте са ширењем интернета ствари (*eng. Internet of Things - IoT*) и порастом броја уређаја који, упркос ограниченим ресурсима, поуздано морају да извршавају све сложенију управљачку логику.

Резултати рада пружају конкретну основу за избор имплементационог приступа машина стања, како избор имплементације не би био заснован на интуицији или стилу, већ на експерименталним резултатима који приказују компромисе између брзине, меморије и прегледности кода.

Рад је организован на следећи начин: Поглавље 2 даје теоријски преглед испитиваних имплементационих приступа машини стања, Поглавље 3 описује методологију мерења и коришћени хардвер, Поглавље 4 представља измерене резултате и њихову анализу, укључујући поређење са теоријским очекивањима из литературе, док Поглавље 5 садржи закључке рада и предлоге за даљи рад.

2. Теоријске основе

Предмет овог рада је поређење осам имплементационих приступа машини стања, од којих је за сваки у наставку рада уведен и доследно коришћен назив на српском језику, уз оригинални (енглески) назив наведен у загради при првом помињању:

1. Угнеждена *Switch-case FSM (Nested Switch — break)*
2. Модификована угнеждена *Switch-case FSM (Nested Switch — return)*
3. *FSM* структурног низа (*Array of Structs*)
4. Директни *LUT FSM (Indexed Table)*
5. *FSM* објекта стања (*State Object*)
6. Планарни *HSM FSM (HSM flat)*
7. Угнеждени *HSM FSM (HSM nested)*
8. Индиректни *LUT FSM (Santic Lookup Table)*

У наставку је дат теоријски преглед свих осам, претходно наведених, имплементационих приступа машина стања. За сваки приступ наведена је релевантна литература која га описује — механизам рада, теоријска оцена сложености, као и познате предности и мане. За сваки од наведених приступа, посебно је размотрено да ли имплементација коришћена у раду верно одговара концепту из цитираних извора или се увиђају било каква одступања. На местима где одступање постоји, то је експлицитно наведено и образложено. Овакав приступ обезбеђује да поређење са резултатима мерења у Поглављу 4 буде прецизно утемељено на ономе што је стварно имплементирано, а не на претпоставци да свака имплементација доследно прати свој извор до најситнијег детаља.

2.1 Угнеждена *Switch-case FSM*

Угнеждени *switch* искази (*eng. nested switch statements*) представљају један од најстаријих и најраспрострањенијих приступа имплементацији машина стања, који су уз каскадне *if* исказе сврстани међу најпопуларније не-објектно-оријентисане имплементације *FSM-a* [РА06]. Овај приступ је у изузетно честој употреби и уз то обезбеђује брзо извршавање, по цени слабије читљивости и одрживости кода.

Структурно, спољашњи *switch* исказ прати тренутно стање као скаларну променљиву, док унутрашњи *switch* исказ прати тренутни догађај, такође као скаларну променљиву [СО23]. Ово представља ефикасно решење за једноставне, планарне (*eng. flat*) аутомате, али без подршке за напреднију функционалност попут хијерархије или историје стања. Пример кода приказан у наставку (Кодни листинг 1) генерише имплементацију у којој се резултат транзиције прво уписује у привремену променљиву, а затим се излази из оба нивоа *switch-a* путем *break* исказа [СО23].

```

...
switch(state)
{
case StateA:
{
    switch(event)
    {
    case Event1:
    {
        nextState = Event1Handler();
        break;
    }
    }
    break;
}
case StateB:
...

```

Кодни листинг 1- Carlgren i Oskarsson угнеждени switch-case [CO23]

Ова структура се поклапа са имплементацијом коришћеном у овом раду (Кодни листинг 2), уз једну разлику: генерисани код у [CO23] сваку транзицију делегира одвојеној *eventHandler* функцији, која у основном облику само враћа следеће стање, али представља место предвиђено за додатну корисничку логику за акцију. Имплементација у овом раду *next_state* уписује директно унутар *case* гране, без тог посредничког позива функције, чиме се избегава додатни трошак позива функције који није предмет мерења. Идентична структура диспечовања (начин на који програм, када добије тренутно стање и тренутни догађај, долази до одлуке која транзиција треба да се изврши). Имплементација *switch(CurrentState)* са унутрашњим грањањем по догађају — заступљена је и у *FSM2Construct* алгоритму [KJH23], што потврђује да се ради о општеприхваћеном приступу, а не о решењу специфичном за један извор.

Ниједан од наведених извора не даје формалну оцену временске сложености угнеженог *switch* приступа, за разлику од *State Table Pattern-a*, за који је експлицитно наведена $O(c)$ сложеност [PA06]. Очекивано понашање зависи од тога да ли компајлер успева да густо попуњен *switch* преведе у табелу скокова (ефективно $O(1)$) или у ланац узастопних поређења ($O(broj_grana)$). Емпиријски је показано да *Nested Switch* приступ при -O2 нивоу оптимизације захтева најмање програмске меморије од свих испитаних приступа, уз напомену да његова читљивост опада са порастом броја стања и догађаја услед све дубље угнежености [CO23].

```

switch (state) {
    case STATE_IDLE:
        switch (event) {
            case EV_VALID:
                next_state = STATE_CHECKING;
                break;
            default:
                next_state = STATE_IDLE;
                break;
        }
        break;
    case STATE_CHECKING:
        switch (event) {
            case EV_VALID:
                next_state = STATE_GRANTED;
                break;
            case EV_INVALID:
                next_state = STATE_DENIED;
                break;
            ...
        }
    return next_state;
}

```

Кодни листинг 2 - Угнеждена Switch-case FSM

2.2 Модификована угнеждена *Switch-case FSM*

Имплементација модификованог угнеженог *Switch-case FSM-a* задржава идентичан механизам диспечовања описан у претходном одељку (спољашњи *switch* по стању, унутрашњи по догађају), уз једну измену: резултат транзиције се враћа директно из сваке гране (*return STATE_X;*), уместо да се прво упише у привремену променљиву па затим врати коришћењем *break* исказа на крају функције.

За разлику од *break* варијанте, која се доследно поклапа са конкретним примером кода (Кодни листинг 1) [CO23], *return* варијанта не одговара доследно ниједном од цитираних извора — представља проширење уведено у овом раду (Кодни листинг 3), ради изолованог мерења утицаја стила писања кода на перформансе, независно од саме алгоритамске структуре диспечовања која остаје идентична *break* варијанти.

Будући да су обе варијанте семантички еквивалентне, кодирају идентичну транзициону табелу, само различитим синтаксним обрасцем, свака измерена разлика у броју циклуса или заузећу меморије између њих може се приписати искључиво начину писања кода, а не разлици у самом алгоритму. Ово ову варијанту чини контролном тачком за проверу да ли, и у којој мери, стилски избор при писању кода утиче на перформансе, питање које ниједан од цитираних извора директно не разматра.

```

...
switch (state) {
    case STATE_IDLE:
        switch (event) {
            case EV_VALID:
                return STATE_CHECKING;
            default:
                return STATE_IDLE;
        }
    case STATE_CHECKING
    ...

```

Кодни листинг 3 – Модификована угнеждена Switch-case FSM

2.3 FSM структурног низа

FSM структурног низа (*eng. Array of Structs* приступ) представља алтернативу угнеженом *switch*-у за имплементацију планарних (*eng. flat*) аутомата: генерише се низ структура које садрже све валидне комбинације стања и догађаја, а индексирање се заснива на енумерисаним типовима [CO23]. Овај приступ заснован је на техници коју први демонстрира пример имплементације аутомата банкомата у програмском језику C [AK17]. Структура коришћеног кода (Кодни листинг 4), садржи показивач на *eventHandler* функцију као треће поље структуре — {стање, догађај, показивач на *handler*} [AK17], [CO23]. Након што линеарна претрага пронађе одговарајући унос, следеће стање се добија позивом те функције, а не директним читањем поља из структуре.

```

...
typedef struct
{
    enumStates      stateMachineState;
    enumEvents      stateMachineEvent;
    eventHandler    stateMachineEventHandler;
} stateMachineStruct;
...
stateMachineStruct stateMachine[] =
{
    {StateA, Event1, Event1Handler},
    {StateA, Event2, Event2Handler}
};
...
nextState =
(*stateMachine[newEvent].stateMachineEventHandler) ();
...

```

Кодни листинг 4 - Carlgren i Oskarsson array of structs implementation [CO23]

У овом раду имплементирани су две варијанте овог приступа. Прва варијанта у потпуности одговара изворном концепту (Кодни листинг 4), која садржи позив *transition_table[i].handler()* [AK17], [CO23], чиме доследно прати изворни механизам (Кодни листинг 5). Друга, поједностављена варијанта коју уводи овај рад, у трећем пољу структуре директно чува вредност следећег стања (Кодни листинг 6), без посредничког позива функције. Ова поједностављена варијанта не одговара ниједном од цитираних извора и уведена је као контролна тачка, како би се приказао допринос појединачне компоненте (позива функције преко показивача) у трошку укупне транзиције, независно од трошка саме линеарне претраге, који је идентичан у обе имплементације.

Будући да је линеарна претрага идентична у обе имплементације, разлика у броју циклуса између њих треба да одговара тачно трошку једног индиректног позива функције — константном, малом износу независном од дужине табеле.

```

typedef struct {
    uint8_t state;
    uint8_t event;
    EventHandler handler; // pokazivac na funkciju, ne
next_state konstanta
} Transition;

static const Transition transition_table[] = {
    { STATE_IDLE,      EV_VALID,    handler_idle_valid },
    { STATE_CHECKING,  EV_VALID,    handler_checking_valid },
    { STATE_CHECKING,  EV_INVALID,  handler_checking_invalid },
    { STATE_GRANTED,   EV_TIMEOUT,  handler_granted_timeout },
    { STATE_DENIED,    EV_TIMEOUT,  handler_denied_timeout },
};

```

Кодни листинг 5 – структурни низ са handler функцијом

```

typedef struct {
    uint8_t state;
    uint8_t event;
    uint8_t next_state;
} Transition;

static const Transition transition_table[] = {
    { STATE_IDLE,      EV_VALID,    STATE_CHECKING },
    { STATE_CHECKING,  EV_VALID,    STATE_GRANTED },
    { STATE_CHECKING,  EV_INVALID,  STATE_DENIED },
    { STATE_GRANTED,   EV_TIMEOUT,  STATE_IDLE },
    { STATE_DENIED,    EV_TIMEOUT,  STATE_IDLE },
};

```

Кодни листинг 6- структурни низ без handler функције

Поређење две имплементације, које се разликују искључиво по томе да ли се следеће стање чита директно из поља структуре или се добија позивом *eventHandler* функције преко показивача (образац који одговара изворном коду (Кодни листинг 4) [AK17], [CO23]), показује на AVR платформи константну, малу разлику у броју циклуса: 12 циклуса на -Os и -O2, и 18 циклуса на -O0. Пошто је линеарна претрага кроз табелу транзиција идентична у обе имплементације, ова разлика произилази искључиво од трошка самог индиректног позива функције. За разлику од стилске разлике између *return* и *break* имплементација угнеженог *switch*-а, која при оптимизацији у потпуности нестаје, ова разлика остаје присутна и на -O2 зато што компајлер не може унапред да зна која се конкретна функција позива преко показивача, па не може да изврши уметање тог позива.

Као представник *FSM* структурног низа приступа, у наставку рада коришћена је поједностављена варијанта, без посредничког позива *eventHandler* функције, што значи да се следеће стање директно чита из структуре.

Handler функција у оригиналном концепту [AK17] , [CO23] , постоји да би извршила неку акцију уз транзицију, док је стање само њена повратна вредност. Пошто *FSM* у овом раду не имплементира никакву додатну акцију у тим функцијама се мери искључиво сама транзиција (*handler* би овде био празна функција, чији би позив само уносио непотребан трошак у мерење). Додатан разлог је избегавање дуплирања исте измерене променљиве: трошак позива функције преко показивача који је засебан предмет мерења код *FSM* објекта стања и *HSM* имплементација. *FSM* структурног низа са *handler*-ом помешао цену линеарне претраге са тим већ измереним трошком, уместо да тестира претрагу као чисту, независну променљиву.

2.4 Директни LUT FSM

State Table Pattern представља решење намењено системима са великим бројем стања и транзиција, посебно у *realtime* и *embedded* окружењима где меморијски *overhead State Design Pattern*-а (заснованог на виртуелним функцијама) није прихватљив [PA06]. Контекст прослеђује догађај *Transition* класи која у константном времену враћа резултујуће стање. У

одељку *Consequences*, наведене су добре перформансе, $O(c)$, као предност, уз два недостатка: табела је скупа за конструкцију и тешка за одржавање, и носи висок иницијализациони трошак (*eng. overhead*), који се може избећи коришћењем *compile-time* конструката (*struct*-ова) као елемената табеле [PA06]. Имплементација у овом раду реализује управо то — табела транзиција је декларисана као *static const*, што значи да је иницијализована у време компајлирања, чиме избегава тај недостатак.

Структурно сродан приступ, познат као *Function Pointers*, представља дводимензиони низ показивача на *eventHandler* функције, чије димензије одређују функције, чије димензије одређују број стања и број догађаја, индексан тренутним стањем и догађајем (Кодни листинг 7).

```
...
static eventHandler StateMachine =
{
    [StateA] = {[Event1] = Event1Handler, [Event2] =
Event2Handler},
    [StateB] = {}
};
...
nextState = (*StateMachine[currentState][newEvent])();
...
```

Кодни листинг 7- Директни LUT Carlgren i Oskarsson [CO23]

Имплементација у овом раду има идентичан облик — индексан дводимензиони низ *[state][event]* — али различит садржај: уместо показивача на функцију, свако поље директно чува вредност следећег стања (*transition_table[state][event]* враћа *next_state* директно, без иједног позива функције). Ово имплементацију чини структурно ближем *Function Pointers* приступу, а функционално ближем *State Table Pattern*-у, што значи да спаја облик једног извора са једноставношћу другог.

Директни *LUT FSM* имплементација у овом раду, приказана у кодном листингу испод (Кодни листинг 8), најтачније одговара *State Table Pattern*-у [PA06] — исти механизам (директно индексирање унапред израчунате табеле, $O(c)$), иста предност (константно време), исти недостаци који су овде избегнути компајл-временском иницијализацијом. Облик табеле (2D низ *[state][event]*) додатно се поклапа са структуром коришћеном за *Function Pointers* приступ [CO23], уз једну суштинску разлику: тај приступ додаје позив функције који Директни *LUT FSM* у овом раду намерно изоставља, чиме се добија чист тест $O(1)$ хипотезе — потврђене у експерименталном делу у наставку, кроз строго константан (*eng. min=avg=max*) број циклуса у сопственим мерењима.

```

static const uint8_t transition_table[NUM_STATES][NUM_EVENTS] =
{
    /*
        EV_VALID          EV_INVALID
    EV_TIMEOUT          */
    /* STATE_IDLE      */ { STATE_CHECKING, STATE_IDLE,
STATE_IDLE            },
    /* STATE_CHECKING */ { STATE_GRANTED,  STATE_DENIED,
STATE_CHECKING        },
    /* STATE_GRANTED   */ { STATE_GRANTED,  STATE_GRANTED,
STATE_IDLE            },
    /* STATE_DENIED    */ { STATE_DENIED,   STATE_DENIED,
STATE_IDLE            },
};

static inline uint8_t fsm_transition(uint8_t state, uint8_t
event) {
    return transition_table[state][event];
}

```

Кодни листинг 8-Директни LUT FSM имплементација

Кодни листинг 8 представља имплементацију коришћену у даљем раду, структурно аналогну генерисаном коду (Кодни листинг 4), уз изостављен позив *eventHandler* функције. Поље низа директно садржи вредност следећег стања.

2.5 FSM објеката стања

Optimal FSM образац [MS02] представља решење за случајеве када је класичан објектно-оријентисани *State Design Pattern* (са виртуелним функцијама и посебном класом по стању) преспоро решење због превеликог броја индиректних позива, што је од посебног значаја за *embedded realtime* системе. Уместо да свако стање буде посебан објекат, стање се своди на показивач на функцију (метод), а тренутно стање је тај показивач [PA06].

У склопу овог приступа истичу се предности које се директно препознају у овој имплементацији: мали меморијски отисак, ефикасна транзиција кроз једну доделу показивача на стање, и једноставност имплементације. Кључна напомена о перформансама односи се на очекивану сложеност $O(\log n)$, где n представља број грана у *switch* исказу — сам прелаз у ново стање је тренутан (једна додела), али обрада догађаја унутар *handler*-а зависи од броја грана којима се испитује који је догађај стигао, па укупна цена транзиције није чисто константна, већ зависи од те унутрашње структуре [PA06]. Даља оптимизација могућа је заменом тог грањања табелом претраге унутар критичних *handler*-а, што је суштински Директни *LUT FSM* приступ примењен локално, унутар једне функције.

```

typedef uint8_t (*StateHandler)(uint8_t event);

static StateHandler const state_handlers[NUM_STATES] = {
    state_idle_handle,
    state_checking_handle,
    state_granted_handle,
    state_denied_handle,
};

static inline uint8_t fsm_transition(uint8_t state, uint8_t
event) {
    return state_handlers[state](event);
}

```

Кодни листинг 9 – Директна LUT FSM имплементација

Сваки елемент кода са Кодни листинг 9 директно реализује по један део описаног механизма [PA06]. Низ *state_handlers[NUM_STATES]* представља реализацију идеје да се стање представи као метода — свако од четири стања је једна функција у овом низу (*state_idle_handle* је стање *IDLE*, не показивач ка неком посебном *IDLE* објекту). Променљива *current_state*, иако технички обично целобројно поље, представља показивач на тренутну методу: комбинација индекса и низа (ако је *current_state == 0*, активна метода је она на позицији 0, *state_idle_handle*) остварује исти ефекат као директан показивач на функцију, само индиректно, преко позиције у низу уместо преко сирове адресе. Позив *state_handlers[state](event)* у *fsm_transition()* затим остварује једну доделу показивача на стање — *handler* функција враћа број следећег стања, који се потом уписује у *current_state* једном доделом, без креирања или уништавања било каквог објекта.

Грађање по догађају, за разлику од избора стања, не дешава се у *fsm_transition()*, него унутар сваког појединачног *handler*-а — на пример, *state_checking_handle* садржи низ од две до три *if* провере којима се испитује који је конкретно догађај стигао, будући да је стање (а тиме и који ће се *handler* позвати) већ одређено пре самог позива. На тај унутрашњи број грана односи се теоријска оцена сложености [PA06]. Цена транзиције зависи од тога колико провера треба извршити унутар *handler*-а док се не пронађе поклапање са пристиглим догађајем. Ово директно објашњава зашто је *FSM* објеката стања имплементација у мерењима овог рада спорија од Директног *LUT FSM*-а (26 наспрам 17 циклуса на *AVR* платформи) упркос мањем меморијском заузећу. Директни *LUT FSM* овај корак грађања потпуно изоставља кроз директно читавање из унапред израчунате табеле, док *FSM* објеката стања мора, након што одабере одговарајући *handler*, још увек линеарно испитати који је догађај у питању.

Овај образац је јефтин по меморији, јер захтева само један низ показивача, али управо због тога није најбрже могуће решење. Ако је брзина приоритет, предлага се додавање табеле претраге (попут Директног *LUT FSM* приступа) унутар самог *handler*-а [PA06]. Ово сугерише да се из овог дизајна не могу добити оба својства истовремено: или се остаје на малом меморијском отиску и прихвата $O(n)$ трошак грађања унутар *handler*-а, или се додаје табела и тиме троши више меморије да би се добила $O(1)$ брзина.

Измерени резултати овог рада приказани у наставку (26 наспрам 17 циклуса, *FSM* објеката стања наспрам Директног *LUT FSM-a*) потврђују овај однос — уштеда меморије *FSM* објеката стања приступа долази уз мерљиву цену у броју циклуса, у складу са теоријским очекивањем [PA06].

2.6 Планарни *HSM FSM*

HSM (*Hierarchical State Machine* — хијерархијски коначни аутомат, у литератури познат и као *statechart*) представља проширење класичног (планарног) коначног аутомата у коме једно стање може да садржи комплетан под-аутомат са сопственим стањима и транзицијама. Концепт је формално уведен као решење за проблем комбинаторне експлозије стања код сложених система [DH87]. За разлику од планарног (*eng. flat*) аутомата, где су сва стања равноправна и независна, *HSM* уводи однос родитељ-дете између стања: када је систем у неком детету (*substate-y*), он је истовремено логички и у његовом родитељу (*superstate-y*), што омогућава да се заједничко понашање више подстања дефинише једном, на нивоу родитеља, уместо да се понавља у сваком детету појединачно.

Овај образац има смисла у системима са много јефтиних транзиција на нивоу детета и мало скупљих транзиција на врху хијерархије [PA06]. Механизам подразумева да контекст представља контејнер конкретних објеката стања, при чему свака конкретна класа стања може имати сопствену машину стања за своја подстања. У делу *Consequences*, предвиђена је и предност и цена оваквог приступа: раздвајање на више нивоа чини сваки ниво једноставнијим, па су транзиције на сваком нивоу јефтиније за израчунавање — али, за разлику од угнеженог *switch-a* (где специјализација захтева преправку целе структуре), *Real-Time State Pattern* захтева измену само погођеног стања. Ово не значи да је хијерархија бржа од *flat* приступа, само да је лакша за проширивање [PA06].

Механизам наслеђивања транзиција (*eng. transition inheritance*) прецизно објашњава технику коју имплементација у овом раду користи: ако се у *superstate-y* дефинише транзиција на неки догађај, она важи за сва његова подстања, осим ако подстање дефинише сопствену, специфичнију транзицију за исти догађај — која тада има предност [YA98].

Пример из области роботике додатно потврђује применљивост овог механизма: аутомат за управљање точковима робота дефинише стање *Move* (кретање) као родитељско стање које садржи подстања *Forward* и *Backward*. Оваква хијерархија поједностављује транзицију ка стању *Stop*, која је дефинисана једном, на нивоу *Move*, и аутоматски важи и за *Forward* и за *Backward*, уместо да се понавља у оба [MV03].

Архитектонски развијенија реализација истог концепта заснива се на приступу који дефинише *Context* структуру за чување активног стања и показивача на *dispatch* функцију која делегира догађаје ка одговарајућем стању, уз посебну *CompositeState* структуру која прати број региона и низове подстања за случајеве са више нивоа угнеђености [SS19], [CO23]. Ова реализација укључује и подршку за *history* стања и *orthogonal* региона, које имплементација у овом раду не користи. *HSM flat/nested* имплементације у овом раду представљају минималну реализацију исте идеје (*handler* + родитељ + резервисана вредност

која означава необрађен догађај), без додатних механизма који овом експерименту нису потребни (history стање, паралелни региони).

Хијарархијска пропација нагоре (*eng. Bubbling*) је понашање у току извршавања које се дешава сваки пут кад стигне догађај који дете не зна да обради. То се дешава при свакој транзицији која то захтева, током рада програма. Променљива *node* у коду само прати где се диспечер тренутно налази док тражи ко зна да одговори на догађај. На почетку је то увек право, тренутно стање аутомата (*node = state*). Ако *handler* тог стања не зна одговор, *node* се помери на његовог родитеља (*node = state_table[node].parent*, као што је приказано у коду - Кодни листинг 10), и провера се понавља тамо. Ово се дешава само унутар једног позива *fsm_transition()* — *node* се после мења и заборавља, док се стварно стање аутомата (*current_state*) не мења све док се не пронађе коначан одговор.

```
static const StateNode state_table[NUM_STATES] = {
    /* STATE_IDLE */ { state_idle_handle, -1
},
    /* STATE_CHECKING */ { state_checking_handle, -1
},
    /* STATE_DENIED */ { state_denied_handle, -1
},
    /* STATE_GRANTED */ { state_granted_handle, -1
},
    /* STATE_NORMAL_ACCESS */ { state_normal_access_handle,
STATE_GRANTED }, // child of GRANTED
    /* STATE_ADMIN_ACCESS */ { state_admin_access_handle,
STATE_GRANTED }, // child of GRANTED
};
static uint8_t fsm_transition(uint8_t state, uint8_t event) {
    int8_t node = state;
    uint8_t result;

    while (node != -1) {
        result = state_table[node].handler(event);
        if (result != EVENT_UNHANDLED) {
            return result;
        }
        node = state_table[node].parent;
    }
    return state;
}
```

Кодни листинг 10 – механизам bubbling-a

Имплементација у овом раду користи генерички диспечерски механизам (*handler* функција по стању + показивач на родитеља + резервисана вредност за необрађен догађај) над *FSM*-

ом са истим скупом стања као у осталим тестираним приступима (*IDLE*, *CHECKING*, *GRANTED*, *DENIED*), али са празним пољем родитеља код сваког стања.

```
// ----- State table: every parent is -1 (NO real hierarchy
in this test) -----
static const StateNode state_table[NUM_STATES] = {
    { state_idle_handle,      -1 },
    { state_checking_handle,  -1 },
    { state_granted_handle,   -1 },
    { state_denied_handle,    -1 },
};

// ----- HSM dispatch: try current state, bubble up to
parent if unhandled -----
static uint8_t fsm_transition(uint8_t state, uint8_t event) {
    int8_t node = state;
    uint8_t result;

    while (node != -1) {
        result = state_table[node].handler(event);
        if (result != EVENT_UNHANDLED) {
            return result; // consumed by this state (or an
ancestor)
        }
        node = state_table[node].parent; // bubble up
    }
    return state; // nobody in the chain handled it -> stay in
current state
}
```

Кодни листинг 11 – Planarni HSM FSM

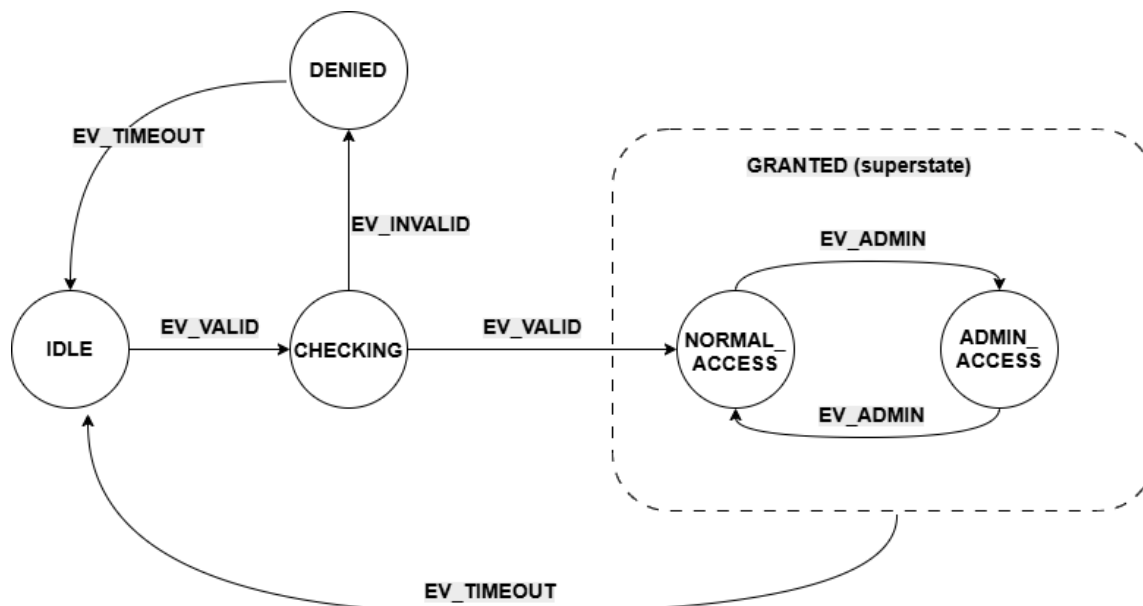
Кодни листинг 11 приказује *Планарну HSM FSM (HSM flat) имплементацију*.

Пошто је *parent* свуда -1, диспечерска петља (`while (node != -1)`) се у пракси никад не пење на други ниво. Сваки *handler* обрађује све своје догађаје сам, без потребе за хијарархијском пропагацијом нагоре. Овај тест не мери корист од хијерархије (које овде нема), већ искључиво фиксни трошак самог диспечерског механизма — петље и провере резервисане вредности — у поређењу са директним приступима попут Директног *LUT FSM-a* или *FSM* објеката стања. Планарни *HSM FSM* представља контролну тачку за Угнеждени *HSM FSM*. Разлика између њих изоловано показује цену стварне хијарархијске пропагације нагоре, одвојену од цене самог механизма.

2.7 Угнеждени HSM FSM

Угнеждени *HSM FSM* користи идентичан диспечерски механизам описан у претходном одељку, али сада са стварним односом родитељ-дете између стања. Прва имплементација у овом раду код које хијарархијске пропагације нагоре описано у 2.6 стварно наступа. Скуп

стања је проширен: *GRANTED* престаје да буде лист и постаје *superstate* који садржи два нова подстања, *NORMAL_ACCESS* и *ADMIN_ACCESS* (уведена је и нова врста догађаја, *EV_ADMIN*, која пребацује између њих). Имплементација приказана на слици (Слика 1) реализована је по узору на механизам наслеђивања транзиција описан у претходном одељку [YA98].



Слика 1 – Модификовани HSM FSM са родитељским стањем

Кодни листинг 13 приказује начин на који *state_granted_handle* обрађује искључиво *EV_TIMEOUT* — ову транзицију не дефинише ни *NORMAL_ACCESS* ни *ADMIN_ACCESS* појединачно, већ је наслеђују од заједничког родитеља *GRANTED*, у складу са механизмом наслеђивања транзиција [YA98]. Свако од два подстања дефинише само своје специфично понашање (*EV_ADMIN*, пребацивање између *NORMAL_ACCESS/ADMIN_ACCESS*), док *EV_TIMEOUT* намерно остаје необрађен на том нивоу и врши хијарархијску пропагацију нагоре.

```

static uint8_t state_granted_handle(uint8_t event) {
    if (event == EV_TIMEOUT) return STATE_IDLE;
    return EVENT_UNHANDLED;
}

static uint8_t state_normal_access_handle(uint8_t event) {
    if (event == EV_ADMIN) return STATE_ADMIN_ACCESS;
    return EVENT_UNHANDLED;
}

```

Кодни листинг 12 - Угнеждени HSM FSM (HSM nested) имплементација

Методологија мерења директно циља овај случај: пре сваке мерене транзиције, стање се експлицитно поставља на *NORMAL_ACCESS*, а примењени догађај је увек *EV_TIMEOUT* —

свака појединачна мерена транзиција тако изолује тачно један корак хијарархијске пропагације нагоре, без мешања са локално обрађеним транзицијама (*EV_ADMIN*). Разлика у броју циклуса између Планарног *HSM FSM-a* (без хијарархијске пропагације нагоре) и Угнеженог *HSM FSM-a* (један корак хијарархијске пропагације нагоре) тиме представља чист, изолован трошак наслеђивања транзиција који литература описује, а овај рад у наставку експериментално доказује.

2.8 Индиректни *LUT FSM* (Santic)

За разлику од осталих извора у овом раду, овај приступ потиче из практичног, инжењерског чланка намењеног *embedded* програмерима [JSnd]. Постоје два уобичајена начина имплементације *FSM-a* у програмском језику *C*: угнеждени *switch* искази, и табела претраге (*eng. lookup table*), која представља концизнији приступ [JSnd].

Коришћени механизам обухвата:

1. Набрајање свих стања и догађаја у енумерисаном типу података.
2. Креирање скупа табела, једна по стању, где сваки унос представља функцију која се извршава за тај пар (стање, догађај).
3. Постављање елемената табеле у јединствен дводимензиони низ индексан стањем и догађајем.

Кључна разлика у односу на остале имплементације са показивачима на функције је што акционе процедуре типа *void* немају повратну вредност. Уместо да транзиција буде изражена кроз оно што функција врати, она се изражава кроз оно што функција уради као споредни ефекат, било да та акција представља подешавање новог стања унутар саме функције, Кодни листинг 14, [JSnd] .

```
void action_sl_el (void)
{
    /* do some processing here */

    current_state = STATE_2;    /* set new state, if necessary
*/
}
```

Кодни листинг 13 - Santic акциона процедура без повратне вредности [JSnd]

Конкретно, акциона процедура сама, унутар тела функције, уписује ново стање у глобалну променљиву (*current_state = STATE_2;*), као што је приказано (Кодни листинг 14), за разлику од имплементације са показивачима функција, где *handler* враћа следеће стање, а упис обавља позивалац споља (Кодни листинг 4), [CO23]. Број индиректних позива функције је идентичан у оба приступа (тачно један по транзицији) — разликује се само механизам преноса резултата (*return* вредност наспрам директног уписа дељене променљиве изнутра).

За разлику од претходно наведене литературе, која се фокусира на перформансе и архитектуру, овде се експлицитно упозорава на безбедносни ризик специфичан за табеларне приступе [JSnd]. Угнеждени *switch* има уграђену заштиту (*default*: грана) која неважећи улаз једноставно игнорише или преусмери на безбедно понашање. Директно индексирање низа (небитно да ли је Директни *LUT FSM* или било која варијанта са показивачима на функције) ту заштиту нема уграђену у сам механизам, и неопходно ју је експлицитно додати као проверу пре приступа низу — иначе ће неважећи индекс довести до недефинисаног понашања (читање ван граница низа, потенцијални скок на произвољну адресу у меморији ако се ради о низу показивача на функције). Ово је корисна, практична напомена коју академски извори не помињу: брзина табеларних приступа има своју цену, а границе ручно мора да провери програмер, јер то механизам диспечовања не ради сам [JSnd].

Имплементација ове методе у даљем раду преузима описани механизам у целости — и акционе процедуре без повратне вредности, и експлицитну проверу граница [JSnd]. То значи да се овај приступ од имплементације са показивачима на функције (Кодни листинг 14) разликује на два начина одједном: по томе како се преноси следеће стање (*return* вредност наспрам уписа изнутра), и по томе што има проверу граница коју она нема [CO23]. Пошто су ова два фактора присутна заједно, из саме теорије не може се рећи колико сваки од њих доприноси евентуалној разлици у броју циклуса.

Зато се у експерименталном делу уводи и треће поређење — Директни *LUT FSM*, који нема ни позив функције ни проверу граница. Поређењем све три имплементације (Директни *LUT FSM*, имплементација са показивачима на функције, *Santic*-ов приступ) раздваја се колико од разлике у циклусима и заузећу меморије долази од самог позива функције, а колико додатно од провере граница [JSnd] .

3. Методологија

У Поглављу 2 представљен је теоријски механизам сваког од осам разматраних имплементационих приступа, укључујући и хијерархијску организацију стања (*superstate/substate* однос, наслеђивање транзиција, *entry/exit* акције) на којој се заснивају Планарни и Угнежђени *HSM FSM*. У наставку је описан конкретан експериментални оквир у коме су сви приступи имплементирани и измерени: заједнички аутомат коришћен за поређење, коришћене хардверске платформе, и механизам мерења заузећа меморије и броја процесорских циклуса по транзицији.

3.1 Елементарна машина стања

Како би се различити стилови и архитектуре имплементације машина стања могли објективно упоредити, дефинисан је један заједнички, једноставан коначни аутомат који је затим имплементиран на осам различитих начина описаних у Поглављу 2 (Угнежђена и Модификована угнежђена *Switch-case FSM*, *FSM* структурног низа, Директни *LUT FSM*, *FSM* објекта стања, Планарни и Угнежђени *HSM FSM*, Индиректни *LUT FSM*), при чему су логика прелаза стања, скуп догађаја и мерна инфраструктура (бројач циклуса преко тајмера, *UART* комуникациони протокол и аутоматизована *benchmark* рутина од 1000 транзиција) идентична у свим приступима осим Планарног и Угнежђеног *HSM FSM-a*, који намерно уводе контролисану измену структуре ради изолованих мерења (детаљно образложено у одељцима 2.6 Планарни *HSM FSM* и 2.8 Индиректни *LUT FSM* (Santic)). На овај начин свака разлика у заузећу меморије или броју извршених циклуса процесора може се приписати искључиво стилу односно архитектури имплементације, а не разлици у самој спецификацији понашања система.

Одабрани тест-пример представља поједностављени модел система контроле приступа (*Access Control FSM*), који моделује типичан сценарио провере пре него што се кориснику дозволи приступ неком ресурсу. Аутомат садржи четири стања:

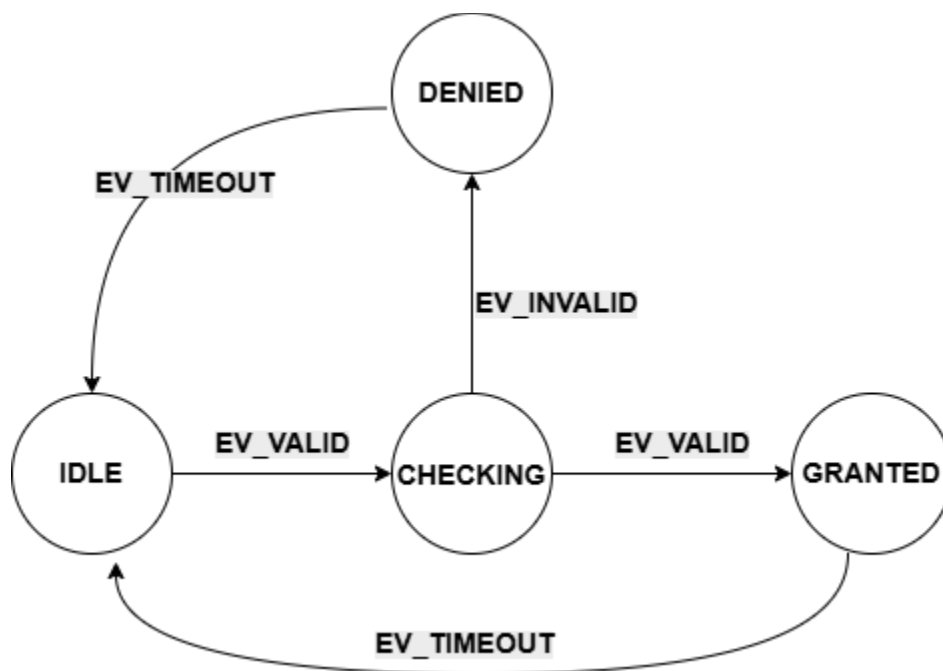
- **IDLE** — почетно стање у коме систем чека захтев за приступ
- **CHECKING** — прелазно стање у коме се врши провера ваљаности унетих акредитација
- **GRANTED** — стање у коме је приступ одобрен
- **DENIED** — стање у коме је приступ одбијен

Прелази између стања дефинисани су са три догађаја:

- **EV_VALID** — сигнализира да су акредитације препознате као исправне
- **EV_INVALID** — сигнализира да су акредитације препознате као неисправне
- **EV_TIMEOUT** — сигнализира истицање временског периода додељеног тренутном стању

Из стања *IDLE*, доласком догађаја *EV_VALID*, аутомат прелази у стање *CHECKING*. Из стања *CHECKING*, аутомат прелази у стање *GRANTED* уколико наступи догађај *EV_VALID*, односно у стање *DENIED* уколико наступи догађај *EV_INVALID*. Из стања *GRANTED* и *DENIED*, аутомат се доласком догађаја *EV_TIMEOUT* враћа у почетно стање *IDLE*, чиме се

затвара циклус и систем постаје спреман за нови захтев за приступ. Графички приказ аутомата дат је на слици (Слика 2).



Слика 2 – дијаграм стања елементарног FSM-а

Овај аутомат је сврсисходно одабран као минималан, али структурно репрезентативан пример: садржи линеаран низ прелаза ($IDLE \rightarrow CHECKING$), грањање на основу исхода провере ($CHECKING \rightarrow GRANTED / DENIED$) и повратне прелазе у почетно стање ($GRANTED \rightarrow IDLE$, $DENIED \rightarrow IDLE$), чиме покрива основне обрасце понашања који се јављају у знатно сложенијим системима, уз задржавање довољно мале величине (четири стања, три догађаја, пет дефинисаних прелаза) да омогући прегледну, ручно проверљиву анализу генерисаног машинског кода за сваку од разматраних имплементација.

3.2 Проширена машина стања

Конкретан модел коришћен у овом раду (*mosquitto_two_client.dot*) преузет је из јавно доступног репозиторијума модела аутомата *Radboud University* [AUTOnD], у оквиру скупа *BenchmarkMQTT*, генерисаних техником активног учења аутомата над пет различитих имплементација *MQTT* брокера [TAB17]. Модел коришћен овде специфично одговара понашању *mosquitto* брокера у сценарију истовремене комуникације два клијента, и садржи шеснаест стања и сто четрдесет четири транзиције, у потпуности преузете без измене [TAB17].

За разлику од елементарне машине стања (одељак 3.1), која је ручно дефинисана за потребе овог рада као минималан, лако проверљив тест-пример, проширена машина стања преузима спецификацију из независног, формалног модела понашања стварног система — *mosquitto*

MQTT брокера. MQTT (eng. *Message Queuing Telemetry Transport*) протокол представља лаган, публикуј-претплати (eng. *publish-subscribe*) протокол за размену порука, широко заступљен у *Internet of Things (IoT)* применама, где уграђени уређаји ограничених ресурса комуницирају преко нестабилних или мрежа ограниченог пропусног опсега. Коришћени протокол је одличан пример комуникације која има карактер коначног аутомата, понашање брокера у сваком тренутку зависи искључиво од тренутног стања везе (успостављена, прекинута, са активним претплатама) и пристиглог догађаја, без потребе за додатном меморијом изван самог стања.

Модел је записан у *.dot* формату — стандардизованом текстуалном запису усмерених графова који користе алати попут *Graphviz* — где сваки чвор представља једно достижно стање система, а свака усмерена грана транзицију изазвану конкретним догађајем. Генерисањем дијаграма машине стања у *Graphviz*-у добијамо нечитљив приказ коришћеног аутомата, па су сви прелази имплементирани на основу *.dot* фајла [AUTond].

Аутомат садржи шеснаест стања (означених S0–S15) и девет типова догађаја:

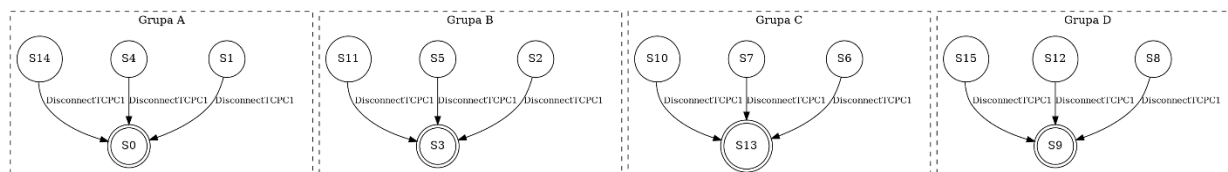
- **ConnectC2, ConnectC1WithWill** → покушај успостављања везе другог, односно првог клијента (при чему C1 везу успоставља уз *Last Will* поруку, MQTT механизам којим брокер, у случају неочекиваног прекида везе, објављује унапред дефинисану поруку у име клијента)
- **PublishQoS0C2, PublishQoS1C1** → објава поруке другог клијента без гаранције испоруке (QoS ниво 0, *at most once*), односно првог клијента уз потврду пријема од стране брокера (QoS ниво 1, *at least once*)
- **SubscribeC1, UnSubscribeC1, SubscribeC2, UnSubscribeC2** → претплата и одјава претплате сваког од два клијента на одређену тему
- **DisconnectTCPC1** → прекид TCP везе инициран од стране првог клијента

За разлику од елементарне машине стања, где транзиције носе искључиво информацију о циљном стању, изворни *.dot* модел уз свако ново стање бележи и стваран одговор посредника (eng. *broker*) на дати догађај, посебно за сваког од два клијента. Иако ова додатна информација није директно искоришћена у диспечерском механизму имплементација тестираних у овом раду (које рачунају искључиво циљно стање), она показује да модел верно одражава протоколарно исправно понашање посредника, а не поједностављену апроксимацију.

Два примера илуструју ову особину модела. При преласку из почетног стања S0 (обе везе затворене) у стање S3, изазваном догађајем *ConnectC2*, изворна ознака бележи да посредник потврђује успостављену везу другог клијента (eng. *ConnAck*), док веза првог клијента остаје затворена. Слично, ако клијент који је већ повезан поново пошаље захтев за конекцију (стање S2, догађај *ConnectC2*), одговор посредника је прекид постојеће везе (*ConnectionClosed*) — понашање специфично за нерегуларан, поновљен захтев. Овакав ивичан случај лако би могао да буде пропуштен у ручно, интуитивно дефинисаном моделу, док га техника активног учења аутомата систематски открива директним испитивањем стварне имплементације.

Пошто транзициона табела проширене машине стања потиче из спецификације генерисане независно од овог рада, исправност сваке имплементације морала је бити засебно верификована пре него што су мерења циклуса и заузећа меморије могла бити сматрана поузданим. Верификација је спроведена програмски — за свих једанаест тестираних

имплементација, свих сто четрдесет четири транзиције генерисане у изворном коду упоређене су, аутоматизовано, са одговарајућим транзицијама из изворне *.dot* спецификације. Тек након потврђеног потпуног функционалног поклапања за сваку имплементацију приступило се мерењима представљеним у Поглављу 4.



Слика 3 – MQTT илустација хијерархија родитељских стања

За разлику од елементарне машине стања, где је хијерархијска варијанта (Угнеждени *HSM FSM*) захтевала увођење новог, вештачког стања *GRANTED* као контејнера за подстања, за проширену машину стања хијерархија је изведена искључиво од постојећих стања аутомата. Анализом *.dot* спецификације утврђено је да се догађај *DisconnectTCPC1* понаша идентично за групе од по три стања (Слика 3), при чему свака група циља исто, већ постојеће стање за које је *DisconnectTCPC1 self-loop*:

Група	Деца (извор bubbling-a)	Циљ на <i>DisconnectTCPC1</i>	Родитељ
A	S1, S4, S14	→ S0	S0 (<i>eng. self-loop</i> , већ тачан циљ)
B	S2, S5, S11	→ S3	S3
C	S6, S7, S10	→ S13	S13
D	S8, S12, S15	→ S9	S9

Табела 1 - хијерархијска организација Угнежденог *HSM FSM*-а за проширену машину стања

Пошто S0, S3, S9 и S13 већ представљају достижна, реална стања аутомата чије је понашање за *DisconnectTCPC1* идентично оном које би требало да наследе њихова деца, она могу директно да послуже као родитељи у диспечерском ланцу, без потребе за увођењем иједног новог, вештачког чвора у табелу стања.

3.3 Комплексна машина стања

3.4 Хардверске платформе

Мерења су извршена на две архитектонски различите платформе: *Arduino Uno*, заснован на 8-битном микроконтролеру *ATmega328P* (*AVR* архитектура) који ради на фреквенцији од 16 MHz, и *ESP32*, заснован на 32-битном *Xtensa LX6* језгру које ради на фреквенцији од 240 MHz, коришћен преко *Arduino-ESP32 framework-a*.

Ове две платформе представљају две различите тачке у распону ресурсно ограничених система. *ATmega328P* располаже свега 2 KB радне и 32 KB програмске меморије и извршава код *bare-metal*, без оперативног система, док *ESP32* располаже знатно већим ресурсима (~320 KB RAM-a) и извршава код под управљањем *FreeRTOS* оперативног система у позадини, што уводи додатне изворе недетерминизма приликом мерења (детаљно ће образложено у одељку 3.3 Методологија мерења циклуса).

За обе платформе, тестирани код приступа хардверским регистрима директно (без *Arduino* библиотека попут *Serial*), а комуникација са рачунаром ради очитавања резултата мерења остварена је преко *UART* протокола на брзини од 9600 bps.

Архитектонска разлика између ова два микроконтролера огледа се и у начину организације меморије, што има директне последице на резултате мерења представљене у Поглављу 4. *ATmega328P* примењује строгу Харвард архитектуру — програмска (*Flash*) и радна (*SRAM*) меморија имају физички одвојене адресне просторе, тако да се подаци смештени у *Flash* меморији (нпр. константне табеле транзиција декларисане као *const*) не могу директно читати истим механизмом као подаци у *SRAM*-у, већ захтевају посебну инструкцију (*LPM*) или употребу *PROGMEM* директиве, у супротном се копирају у *SRAM* при покретању програма. *ESP32*, заснован на *Xtensa LX6* језгру, примењује модификовану *Harvard* архитектуру — на нивоу хардвера и даље постоје одвојене магистрале за инструкције и податке ради перформанси, али јединица за управљање меморијом (*Memory Management Unit, MMU*) мапира *Flash* меморију у јединствен, кеширан адресни простор доступан програму, чиме се константни подаци могу читати директно из *Flash-a* (*execute-in-place, XIP*), без потребе за копирањем у *RAM* или употребом посебних инструкција [ESnd] .

Ова разлика у меморијском моделу директно утиче на интерпретацију резултата у Поглављу 4. Идентичан изворни код, чија се величина статичке табеле мења, производи мерљиву *SRAM* разлику на *ATmega328P* платформи (услед строге поделе адресних простора), док на *ESP32* платформи иста промена остаје видљива искључиво у заузећу *Flash* меморије, не и *RAM-a*.

3.5 Методологија мерења циклуса

Поред заузећа меморијских ресурса, за анализе спроведене у овом раду од суштинске важности је и број циклуса процесора утрошених на прелазе између стања *FSM-a*. За рад са регистрима коришћеним у функцијама за мерење перформанси коришћен је званични *datasheet ATmega328P* микроконтролера.

3.5.1 Зашто није коришћен ISR приступ?

У овом раду није коришћен стандардан приступ заснован на прекидним рутинама (*Interrupt Service Routine*, ISR) за сам механизам мерења, јер такав приступ не би био довољно поуздан за мерење броја циклуса везаних за прелазе стања. Прекидна рутина уводи неколико извора недетерминизма који су неприхватљиви када је циљ измерити тачан, поновљив број циклуса процесора утрошених на једну транзицију *FSM-a*:

- **Латенција уласка у прекид** → Од тренутка када се испуни услов за прекид до тренутка када процесор стварно почне извршавати код *ISR-a* пролази променљив број циклуса, јер процесор прво мора завршити тренутно извршавану инструкцију, а затим извршити интерну процедуру скока на вектор прекида и аутоматско чување стања (регистар *SREG*, повратна адреса на стеку). Ово кашњење само по себи уноси шум у мерење који нема везе са ценом саме *FSM* транзиције.
- **Могућност угнеждених или одложених прекида** → Ако се у тренутку окидања прекида процесор налази у другој, већ активној прекидној рутини, или је глобални бит прекида (*I-bit* у *SREG*) привремено искључен (нпр. унутар *cli()/sei()* блока), обрада прекида се одлаже за непознат, променљив број циклуса — што би мерење учинило непоновљивим.
- **Додатни трошкови уласка/изласка из *ISR-a*** → Сам улазак и излазак из прекидне рутине (чување и враћање регистара, извршавање инструкције *RETI*) троши фиксан, али незанемарљив број циклуса који би се, уколико би се ISR користио за ограничавање мерног прозора, неизбежно помешао са бројем циклуса који се жели приписати искључиво функцији *fsm_transition()*.

Ова три извора недетерминизма директно су документована у техничкој спецификацији коришћеног микроконтролера [AMnd] . Према том документу, минимално време одзива на прекид износи четири циклуса такта, при чему се оно увећава уколико се прекид јави током извршавања вишециклусне инструкције, јер се та инструкција мора прво завршити пре него што се прекид опслужу — ово је извор варијабилне латенције уласка у прекид. Надаље, уласком у прекидну рутину бит за глобалну дозволу прекида (*I-бит* у регистру *SREG*) се аутоматски брише, чиме се сви наредни прекиди одлажу све док се тај бит експлицитно не постави — било инструкцијом *RETI* при повратку из прекида, било ручно унутар саме рутине ради омогућавања угнежђених прекида — што уводи могућност непоновљивог кашњења обраде ако се прекиди временски поклопе. Коначно, сам повратак из прекидне рутине (инструкција *RETI*) захтева фиксна четири циклуса такта, док регистар статуса *SREG* није аутоматски сачуван нити враћен при уласку/изласку из прекида — то мора бити експлицитно урађено у софтверу, чиме се додатни циклуси троше на чување/враћање контекста који немају везе са мереном логиком.

Из ових разлога, мерење броја циклуса једне *FSM* транзиције захтева механизам чије је време окидања и трајање потпуно детерминистичко и синхроно са током главног програма — што *polling* приступ, за разлику од *ISR-a*, директно обезбеђује

3.5.2 Polling приступ коришћен за мерење

Уместо *ISR-a*, мерење се заснива на директном, синхронном читању вредности бројача тајмера (*polling*), непосредно пре и непосредно после позива функције *fsm_transition()*.

Овај приступ не генерише никакав прекид нити зависи од било каквог асинхроног догађаја — бројач тајмера се чита директно, у тренутку када то програм експлицитно затражи, чиме се елиминишу сви претходно наведени извори недетерминизма. Додатно, читав мерни прозор (позив *cycles_start()*, извршавање *fsm_transition()*, позив *cycles_stop()*) обавијен је паром *cli()/sei()* инструкција, које привремено онемогућавају све маскиране прекиде (брисањем глобалног *I*-бита у регистру *SREG*) за време трајања мерења. Тиме се спречава да евентуални други, неповезан прекид (нпр. *UART* пријем) прекине мерење усред извршавања и унесе додатне, нежељене циклусе у резултат.

ATmega328P располаже са три тајмера/бројача: 8-битним *TimerCounter0*, 16-битним *TimerCounter1* и 8-битним *TimerCounter2*. За потребе мерења броја циклуса по транзицији одабран је *TimerCounter1*, из следећих разлога:

- **Опсег бројача** → *TimerCounter1* је једини од три тајмера са пуном 16-битном ширином бројача (*TCNT1*, опсег 0–65535), док су *TimerCounter0* и *TimerCounter2* ограничени на 8 бита (*TCNT0/TCNT2*, опсег 0–255). Иако су појединачне *FSM* транзиције у разматраним имплементацијама довољно кратке (типично испод стотинак циклуса) да би стале и у 8-битни опсег, коришћење 16-битног бројача оставља резерву за имплементације са знатно више циклуса по транзицији, без ризика од прекорачења опсега бројања (*overflow*) током једног мерења.
- **Одвојеност од *Timer0*** → У делу анализе који испитује независност цене транзиције од дужине претходног боравка у стању, *Timer0* је наменски употребљен за симулацију трајања стања у *CTC* режиму рада уз прекидну рутину (*ISR(TIMER0_COMP_vect)*), која генерише ~3-секундно кашњење пре него што се изазове мерена транзиција. Када би се за мерење циклуса транзиције користио исти тајмер који истовремено генерише ово кашњење, два механизма би делила исте регистре, што би захтевало пажљиво чување и обнављање конфигурације тајмера пре и после сваког мерења и унело додатни ризик од међусобног ометања. Коришћењем физички одвојене хардверске јединице (*TimerCounter1* за мерење циклуса, *TimerCounter0* за симулацију трајања стања) ова два механизма су потпуно независна.

3.5.3 Регистри *Timer1* коришћени за мерење

Регистар *TCCR1B* (*Timer/Counter1 Control Register B*) је 8-битни регистар који садржи три *Clock Select* бита, *CS12:CS11:CS10* (битови 2:1:0), који бирају извор такта за *Timer1* [AMnd].

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{IO} /1 (no prescaling)
0	1	0	clk _{IO} /8 (from prescaler)
0	1	1	clk _{IO} /64 (from prescaler)
1	0	0	clk _{IO} /256 (from prescaler)
1	0	1	clk _{IO} /1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Слика 4- Табела са вредностима прескалера за *Timer1* [AMnd]

Функција за почетак мерења (*cycles_start()*) прво поставља *TCCR1B* = 0, чиме се сви *clock-select* битови постављају на 0 и тајмер зауставља (нема такта, бројач се не помера). Затим се *TCNT1* = 0 уписује директно у *Timer/Counter1* бројачки регистар, ресетујући га на почетну вредност — пошто је *TCNT1* 16-битни регистар (мапиран преко *TCNT1H/TCNT1L* пара), *C* компајлер аутоматски генерише исправан двобајтни приступ (Кодни листинг 18).

```
static inline void cycles_start(void) {

    TCCR1B = 0;
    TCNT1 = 0;
    TCCR1B = (1 << CS10);

}
```

Кодни листинг 14 – Функција која означава почетак бројања циклуса помоћу регистра *Timer1*

На крају се *TCCR1B* = (1 << CS10) поставља *CS10* = 1, *CS11* = 0, *CS12* = 0, што означава *clk_IO* без прескалирања (Слика 4) — тајмер сада броји на пуној фреквенцији *CPU* такта (16 MHz), што значи да један *tick* одговара тачно једном *CPU* циклусу. Ово је кључно за мерење где бројач директно представља број протеклих *CPU* циклуса, без потребе за конверзијом.

```
static inline uint16_t cycles_stop(void) {

    TCCR1B = 0;
    return TCNT1;

}
```

Кодни листинг 15 - Функција која означава завршетак бројања циклуса помоћу регистра *Timer1*

Кодни листинг 19 приказује функцију за крај мерења (*cycles_stop()*) и поново поставља *TCCR1B = 0*, чиме се тајмер зауставља и спречава да бројач настави да расте док се вредност чита или док се извршава неки други код. На крају, *return TCNT1* чита тренутну вредност 16-битног бројача и враћа је као резултат — пошто је тајмер стартован на 0 и бројао без прескалирања тачно онолико циклуса колико је прошло између *cycles_start()* и *cycles_stop()*, ова вредност јесте број *CPU* циклуса који су протекли у том интервалу.

3.5.4 Заједничка функција за мерење транзиције

Функција *measured_transition()*, Кодни листинг 20, представља заједничку обавијајућу функцију (*wrapper function*) која се користи у свих осам имплементација и чији је задатак да изолује тачан број циклуса процесора утрошених на позив функције *fsm_transition()* — конкретна имплементација ове функције разликује се од примера до примера (угнеждени *switch*, линеарна претрага, индексирана табела, диспечовање преко показивача на функцију или *HSM* механизам са хијарархијском пропагацијом нагоре), док логика око ње остаје непромењена.

```
static uint16_t measured_transition(uint8_t event) {  
  
    cli();  
    cycles_start();  
    uint8_t next = fsm_transition(current_state, event);  
    uint16_t cycles = cycles_stop();  
    sei();  
    current_state = next;  
    return cycles;  
  
}
```

Кодни листинг 16 – заједничка функција за мерење броја циклуса потребних за прелазе између стања

Пре позива *fsm_transition()* извршава се инструкција *cli()*, која брише глобални бит за омогућавање прекида (*I*-бит у регистру *SREG*) и тиме онемогућава све маскирајуће прекиде за време трајања мерног прозора, спречавајући да неки неповезан прекид унесе додатно, променљиво кашњење у мерење. Након завршетка мерења позива се *sei()*, која тај бит поново поставља и враћа прекидни систем у нормалан рад. Између ова два позива налазе се *cycles_start()* и *cycles_stop()*, које мере тачан број протеклих циклуса око позива *fsm_transition()*, док се ажурирање глобалног стања (*current_state = next*) намерно извршава тек након што је мерење већ завршено, како не би утицало на измерену вредност.

3.6 Нивои оптимизације компајлера

Према званичној *GCC* документацији [GNUnd] и стандардној литератури за микроконтролере, нивои оптимизације представљају унапред дефинисане скупове оптимизационих пролаза компајлера. Кроз ове оптимизационе прекидаче преводиоца (*eng*.

flag), компајлер прави компромис између три фактора: брзине извршавања, заузећа меморије (*Flash/RAM*) и могућности дебаговања.

-O0 — Без оптимизације (*eng. No optimization / Debugging baseline*)

Ово подешавање искључује практично све оптимизационе алгоритме у компајлеру. Циљ је најбрже могуће време превођења (*eng. compilation time*) и очување тачне структуре кода ради једноставног дебаговања преко *GDB*-а или хардверских емулятора. Свака променљива се строго чува и чита са стека или из *RAM*-а, а редослед инструкција се 1:1 поклапа са изворним *C* кодом. Утицај на микроконтролер (*AVR*): знатно веће заузеће *Flash* меморије и приметно већи број циклуса, односно спорије извршавање.

-Os — Оптимизација за величину (*eng. Optimize for size*)

Ово је подразумевани ниво оптимизације за већину *embedded* платформи, укључујући *Arduino core [MCnd]*. Укључује све оптимизације из *-O2* које не повећавају величину бинарног кода. Циљ је минимално заузеће *Flash* и програмског простора на меморијски ограниченим микроконтролерима. Искључују се технике попут агресивног одмотавања петљи (*eng. loop unrolling*), поравнања функција и блокова (*eng. function/loop alignment*) или агресивног уграђивања функција (*eng. inlining*) уколико оне генеришу више бајтова машинског кода. Утицај на микроконтролер (*AVR*): најмањи *.hex* фајл и најмање заузеће програмске меморије уз веома добре брзинске перформансе.

-O2 — Оптимизација за перформансе / брзину (*eng. Moderate-to-high speed optimization*)

Ово подешавање укључује готово све подржане оптимизације које не подразумевају превелики компромис у величини кода. Циљ је максимално убрзање рада процесора и смањење броја циклуса по инструкцији/функцији. Активирају се напредни пролази попут елиминације заједничких подизраза (*eng. Global Common Subexpression Elimination, GCSE*), анализе тока података, елиминације мртвог кода и оптимизације скокова и грањања (*thread jumps, crossjumping*). Утицај на микроконтролер (*AVR*): убрзава прелазе стања у *FSM*-у и смањује број циклуса, али често резултује нешто већим *Flash* бинарним фајлом у односу на *-Os*.

4. Резултати мерења и дискусија резултата

У овом поглављу приказани су резултати мерења за свих осам имплементационих приступа уведених у Поглављу 2, добијени применом методологије описане у Поглављу 3. Резултати су организовани у пет одељака: одељак 4.1 приказује измерено заузеће меморије и број процесорских циклуса за елементарну машину стања на *AVR* платформи; одељак 4.2 упоређује ове резултате са теоријским тврдњама из релевантне литературе; одељак 4.3 приказује резултате истих мерења на *ESP32* платформи; одељак 4.4 даје упоредну анализу перформанси и заузећа ресурса између две платформе; одељак 4.5 засебно разлаже трошак индиректних приступа диспечовању на компоненту позива функције преко показивача и компоненту провере граница. Резултати приказани у овом поглављу односе се на елементарну машину стања (*eng. Access Control FSM*, четири стања), као и резултати за проширену (*eng. Mosquitto Two Client Protocol MQTT*) и комплексну машину стања.

4.1 Резултати на елементарној машини стања

4.1.1 AVR платформа

Сва мерења у овом одељку изведена су на платформи Arduino Uno (*ATmega328P*, 16 MHz), према методологији описаној у Поглављу 3 Методологија мерења, за осам именованих приступа уведених у Поглављу 2 Теоријске основе.

Начин имплементације	Flash - O0	Flash - Os	Flash - O2	SRAM - O0	SRAM - Os	SRAM - O2
Модификовани угнеждени <i>switch-case FSM</i>	2512 bytes	1476 bytes	1928 bytes	286 bytes	290 bytes	290 bytes
Угнеждени <i>switch-case FSM</i>	2582 bytes	1476 bytes	1928 bytes	286 bytes	290 bytes	290 bytes
<i>FSM</i> пиза структура	2518 bytes	1494 bytes	1952 bytes	304 bytes	308 bytes	308 bytes
Директни <i>LUT FSM</i>	2438 bytes	1526 bytes	1978 bytes	298 bytes	302 bytes	302 bytes
Индиректни <i>LUT FSM (Santic)</i>	2642 bytes	1568 bytes	2090 bytes	340 bytes	346 bytes	346 bytes
<i>FSM</i> објеката стања	2562 bytes	1520 bytes	2008 bytes	292 bytes	298 bytes	298 bytes
Планарни <i>HSM FSM</i>	2640 bytes	1564 bytes	2024 bytes	302 bytes	308 bytes	308 bytes

Начин имплементације	Flash - O0	Flash - Os	Flash - O2	SRAM - O0	SRAM - Os	SRAM - O2
Угнеждени <i>HSM FSM</i>	2796 bytes	1688 bytes	2048 bytes	410 bytes	418 bytes	418 bytes

Табела 2- Резултати мерења, заузеће Flash/SRAM меморије код ATmega328P микроконтролера

Заузеће радне меморије (SRAM) остаје стабилно за сваку имплементацију независно од нивоа оптимизације, док заузеће Flash меморије опада са -O0 на -Os, а затим благо расте на -O2. Овај образац је доследан кроз свих осам приступа.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угнеждени <i>switch-case FSM</i>	78	81	84
Угнеждени <i>switch-case FSM</i>	90	92	95
<i>FSM</i> пиза структура	121	160	211
Директни <i>LUT FSM</i>	79	79	79
Индиректни <i>LUT FSM (Santic)</i>	82	82	83
<i>FSM</i> објеката стања	105	105	105
Планарни <i>HSM FSM</i>	134	134	134
Угнеждени <i>HSM FSM</i>	213	213	213

Табела 3 – Мерења броја циклуса елементарног FSM-а са оптимизацијом -O0

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угнеждени <i>switch-case FSM</i>	13	13	16
Угнеждени <i>switch-case FSM</i>	13	13	16

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
<i>FSM</i> пиза структура	25	42	64
Директни <i>LUT FSM</i>	17	17	17
Индијектни <i>LUT FSM (Santic)</i>	38	38	39
<i>FSM</i> објеката стања	26	26	26
Планарни <i>HSM FSM</i>	37	37	39
Угнеждени <i>HSM FSM</i>	72	72	72

Табела 4 - Мерења елементарног *FSM*-а са оптимизацијом -*Os*

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификовани угнеждени <i>switch-case FSM</i>	13	13	16
Угнеждени <i>switch-case FSM</i>	13	13	14
<i>FSM</i> пиза структура	25	39	59
Директни <i>LUT FSM</i>	18	18	18
Индијектни <i>LUT FSM (Santic)</i>	37	37	38
<i>FSM</i> објеката стања	24	24	25
Планарни <i>HSM FSM</i>	36	37	39
Угнеждени <i>HSM FSM</i>	71	71	71

Табела 5 - Мерења елементарног *FSM*-а са оптимизацијом -*O2*

4.1.2 Поређење AVR резултата са литературом

На нивоу -O0, Угнеждена *Switch-case FSM (break)* заузима 2582 бајта програмске меморије, док Модификована угнеждена *Switch-case FSM (return)* заузима 2512 бајтова — разлика од 70 бајтова (2,71%), искључиво у *Flash* меморији (*SRAM* идентичан, 286 бајтова), Табела 2. На -Os и -O2, ова разлика у потпуности нестаје (идентичних 1476/1928 бајтова, и идентичан број циклуса на оба нивоа осим у максималним вредностима) — последица елиминације мртвих уписа (*eng. dead store elimination*) коју *GCC* примењује на овим нивоима, чиме се две синтаксно различите, семантички еквивалентне имплементације свODE на идентичан облик пре генерисања машинског кода.

Директни *LUT FSM* показује идентичних 17 циклуса у сва три статистичка показатеља на -Os (18 на -O2), Табела 4 и Табела 5 — потпуна одсутност варијације директно потврђује теоријску $O(c)$ тврдњу повезану са *State Table Pattern*-ом [PA06]. За разлику од овога, *FSM* структурног низа показује мерљив распон (25–64 циклуса на -Os), јер број извршених поређења зависи од позиције тражене транзиције у низу.

FSM објекта стања је истовремено меморијски јефтинији (1520 наспрам 1526 бајта на -Os) и спорији (26 наспрам 17 циклуса) од Директног *LUT FSM*-а — потврђује $O(\log n)/O(n)$ оцену изнету у одељку 2.5 [PA06].

Угнеждени *HSM FSM* показује највеће заузеће *Flash* меморије на сва три нивоа оптимизације (2796/1688/2048 B) и истовремено највећи број циклуса (213/72/71), чиме доследно заузима последње место по оба критеријума, Табела 2. Планарни *HSM FSM* (контролна варијанта без стварне хијерархије) показује знатно мање циклусе (134/37/37) — разлика између ове две имплементације (нпр. 72 наспрам 37 на -Os, готово дупло) представља изолован трошак једног корака хијерархијске пропагације нагоре, потврђен емпијски.

Индиректни *LUT FSM (Santic LUT)* показује 82/38/37 циклуса — спорији од Директног *LUT FSM*-а (17/18), али у истом опсегу као *FSM* структурног низа. Детаљно разлагање овог трошка на компоненту саме индирекције и компоненту провере граница дато је у одељку 4.1.5.

Резултати приказани у Табелама 1–4 могу се систематски упоредити са налазима из два независна извора у области образаца пројектовања машина стања [CO23], [PA06].

Поклапање је најизраженије за хијерархијске имплементације. Угнеждени *HSM FSM* у приложеним табелама показује највеће заузеће програмске меморије на сва три испитана нивоа оптимизације и истовремено највећи број циклуса процесора по транзицији, чиме доследно заузима последње место по оба критеријума. Овај налаз директно потврђује да *Hierarchical State Pattern* имплементација доследно показује највеће заузеће меморије и најлошије резултате у погледу времена извршавања и скалабилности са порастом броја стања [CO23], као и да механизам подршке за угнеђена стања уводи додатну структурну сложеност чак и када се та додатна структура у конкретном случају не искоришћава у пуној мери [PA06].

Додатно поклапање уочено је код Директног *LUT FSM*-а, чији је број циклуса процесора идентичан у сва три статистичка показатеља на сваком испитаном нивоу оптимизације, чиме се потврђује теоријска тврдња о константној временској сложености $O(c)$ приписаној *State Table Pattern*-у [PA06].

На нивоу -O2, налаз да Угнеждена *Switch-case FSM* захтева најмање програмске меморије поклапа се са *case-study* резултатом према коме модификовани угнеждени *Switch-case* приступ захтева најмање меморије при истом нивоу оптимизације [CO23].

Два налаза се, међутим, разилазе од литературе. Прво, на нивоу -O0, *FSM* структурног низа не показује ни најмање заузеће меморије ни најкраће време извршавања — трећи је по оба критеријума, иза Угнеждене *Switch-case FSM* и Директног *LUT FSM*-а — док *case-study* без оптимизационих флегова у литератури наводи супротно [CO23]. Објашњење лежи у разлици величине испитиваног аутомата: литература испитује генерисане аутомате са до 1000 стања, док аутомат коришћен у овом раду има свега четири стања и пет дефинисаних транзиција — на тој величини фиксни трошак петље за линеарну претрагу надмашује уштеду коју би та претрага иначе донела. Ово је потврђено експерименталним резултатима у поглављу 4.2.

Друго, *HSM* имплементације у овом раду захтевају више програмске меморије од Директног *LUT FSM*-а на сва три нивоа оптимизације, док литература у општем делу поређења наводи да табеларни приступи захтевају највише меморије од свих шест испитаних приступа [CO23]. Ова разлика се такође може приписати величини аутомата: заузеће меморије табеларног приступа расте пропорционално производу броја стања и броја догађаја ($O(N \times M)$), док фиксни трошак хијерархијске инфраструктуре расте спорије, приближно линеарно са бројем стања.

4.1.3 ESP32 платформа

Директно поређење сирових резултата мерења између *AVR* и *ESP32* платформе показује наизглед огромну разлику у заузећу меморије. За Директни *LUT FSM* са оптимизацијом -O2, на пример, измерено заузеће износи 1978 В наспрам 271 537 В за *Flash*, односно 302 В наспрам 21 472 В за *RAM*. Посматрано изоловано, ово би сугерисало да је идентичан *FSM* код на *ESP32* платформи стотинама пута већи него на *AVR* платформи — закључак који није веродостојан с обзиром да се ради о функционално еквивалентној имплементацији истог алгорита.

Претпоставка је да разлика не потиче од саме *FSM* имплементације, већ од фиксног улазног трошка који *Arduino-ESP32* развојни оквир уноси у сваки *build*, независно од количине корисничког кода — *FreeRTOS scheduler*, иницијализација два језгра, и знатно обимнија имплементација стандардне *C* библиотеке у односу на *AVR toolchain*. Да би се ова претпоставка проверила, у скуп мерења додат је референтни (*eng. baseline*) *sketch* без иједне линије *FSM* кода, чија је измерена вредност затим одузета од сваког резултата, како би се изоловало стварно заузеће самог *FSM* кода.

Табела 6 приказује измерено заузеће празног референтног *sketch*-а на сва три испитана нивоа оптимизације.

Оптимизација	Flash (B)	RAM (B)
-O0	284 017	21 464
-Os	269 441	21 464
-O2	270 653	21 464

Табела 6- заузеће меморије празног (*baseline*) *sketch*-а на *ESP32* платформи

Ове вредности представљају између 99,3% и 99,7% укупног заузећа меморије измереног код сваке *FSM* имплементације, што потврђује да фиксни трошак *Arduino-ESP32* оквира доминира над укупним отиском програма, док сама *FSM* логика чини занемарљив део.

Табела 7 приказује нето заузеће меморије по *FSM* приступу, добијено одузимањем *baseline* вредности од сваког сировог резултата.

Приступ	Flash нето - O0 (B)	Flash нето - Os (B)	Flash нето - O2 (B)	RAM нето (B)
Директни LUT FSM	1432	800	884	0–8
Модификована угнеждена Switch-case FSM (return)	1508	832	932	0–8
Угнеждена Switch-case FSM (break)	1592	816	932	0–8
FSM структурног низа	1540	824	880	0–8
FSM објеката стања	1640	908	960	0–8
Планарни HSM FSM	1752	956	1012	0–8
Угнеждени HSM FSM	1920	1148	1200	0–8
Угнеждени LUT FSM (Santic)	1672	1012	1096	0–8

Табела 7 - нето заузеће меморије по *FSM* приступу на *ESP32* платформи после одузимања *baseline* вредности

Нето разлика у заузећу *Flash* меморије између најкомпактније имплементације (Директни LUT FSM, 800 B на -Os) и најзахтевније (Угнеждени HSM FSM, 1920 B на -O0) износи приближно 1120 B — разлика реда величине десетина процената, не стотина пута, како су то сугерисали сирови подаци. Заузеће *RAM* меморије је практично занемарљиво за све имплементације, са разликом од највише 8 бајтова у односу на *baseline*, независно од сложености *FSM* приступа.

Мерење празног референтног *sketch*-а и одузимање његове вредности од сваког резултата потврђује полазну претпоставку: огромна разлика у апсолутним бројевима између *AVR* и *ESP32* платформе не одражава разлику у ефикасности *FSM* имплементација, већ разлику у величини развојних оквира и стандардних библиотека које свака платформа уноси. Када се овај фиксни трошак изолује, нето заузеће меморије по *FSM* приступу на *ESP32* платформи постаје упоредиво по реду величине са *AVR* платформом, а релативни однос између појединачних приступа (нпр. да је Директни LUT FSM доследно најкомпактнији, а Угнеждени HSM FSM најзахтевнији) остаје конзистентан на обе платформе. Овим се потврђује да су закључци о релативној ефикасности *FSM* приступа независни од избора хардверске платформе, док апсолутне вредности увек морају бити тумачене у контексту фиксног трошка развојног оквира.

Табела 8 приказује измерене вредности на нивоу -O0.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	84	92	5807
Угнеждена Switch-case FSM (break)	96	106	7739
FSM структурног низа	118	169	6782
Директни LUT FSM	74	78	4840
FSM објеката стања	102	110	5832
Планарни HSM FSM	130	141	8712
Угнеждени HSM FSM	205	216	11642
Угнеждени LUT FSM (Santic)	87	93	3885

Табела 8 - број процесорских циклуса по транзицији на ESP32 платформи, -O0

Табела 9 приказује измерене вредности на нивоу -Os.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	21	25	1934
Угнеждена Switch-case FSM (break)	21	25	1934
FSM структурног низа	34	45	1942
Директни LUT FSM	18	19	1932
FSM објеката стања	28	33	2898
Планарни HSM FSM	39	45	3883
Угнеждени HSM FSM	72	76	4858
Угнеждени LUT FSM (Santic)	39	43	3871

Табела 9 - број процесорских циклуса по транзицији на ESP32 платформи, -Os

Табела 10 приказује измерене вредности на нивоу -O2.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	17	22	1938
Угнеждена Switch-case FSM (break)	17	22	1938
FSM структурног низа	25	37	2895
Директни LUT FSM	14	15	1930

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
FSM објеката стања	27	33	3862
Планарни HSM FSM	39	45	2912
Угнеждени HSM FSM	71	78	7727
Угнеждени LUT FSM (Santic)	37	42	3873

Табела 10 - број процесорских циклуса по транзицији на ESP32 платформи, -O2

Поређење имплементација Модификоване угнеждене *Switch-case FSM* и Угнеждене *Switch-case FSM*), које се функционално разликују искључиво по томе да ли се излазна вредност транзиције директно враћа (*eng. return*) или прво уписује у привремену променљиву па затим враћа на крају функције (*eng. break*), показује на ESP32 платформи идентичан образац уочен раније на AVR-у. При искљученој оптимизацији (-O0) разлика између два стила је мерљива (84 наспрам 96 циклуса), јер се додатна инструкција уписа у привремену променљиву дословно извршава. При укљученој оптимизацији, разлика у потпуности нестаје, оба стила дају идентичних 21 циклус на -Os и 17 циклуса на -O2, јер компајлер препознаје привремену променљиву као сувишну и уклања је (*eng. dead store elimination*), свдећи обе имплементације на идентичан машински код. Значај овог налаза није у понављању већ утврђеног резултата, већ у томе што је иста појава сада потврђена на две архитектонски потпуно различите платформе (8-битни AVR и 32-битни Xtensa), чиме престаје да буде специфичност једног компајлерског *toolchain*-а и постаје генерализована особина оптимизационих пролаза заснованих на елиминацији мртвог кода.

За разлику од стилских разлика, разлике које проистичу из саме алгоритамске структуре диспечовања не нестају ни на највишем испитаном нивоу оптимизације. Директни LUT FSM задржава најмањи број циклуса на сваком нивоу (74/18/14 на -O0/-Os/-O2), будући да се своди на једно индексирање дводимензионог низа које компајлер не може даље да скрати. FSM структурног низа, који се ослања на линеарну претрагу кроз табелу транзиција, остаје мерљиво спорији (118/34/25) на сваком нивоу оптимизације, јер чак и када је петља у потпуности уметнута, и даље мора извршити бар једно поређење по елементу табеле, трошак који је последица алгоритма, не стила писања кода, па га оптимизација не може уклонити. Најизраженија разлика уочена је код хијерархијских имплементација: Угнеждени HSM FSM захтева два узастопна индиректна позива кроз показивач на функцију (први ка *handleru* тренутног подстања које догађај не обрађује, други ка *handleru* родитељског стања које га обрађује), и остаје најспорија имплементација на сваком нивоу оптимизације (205/72/71), приближно двоструко спорија од Планарног HSM FSM-а (130/39/39), који извршава само један такав позив. Пошто компајлер не може унапред да зна која се конкретна функција позива кроз показивач, не може ни да изврши уметање (*eng. inlining*) тог позива, па трошак индиректног позива остаје присутан независно од степена оптимизације — налаз који директно потврђује да цена пробубљавања по нивоу хијерархије није аномалија неоптимизованог кода, већ суштинско својство механизма.

За разлику од минималних вредности, које су глатке и предвидиве, максималне измерене вредности повремено достижу и по неколико хиљада циклуса (нпр. за Угнеждени HSM FSM на -O0: минимум 205, максимум 11 642). Ова појава није последица тестиране FSM

имплементације, већ окружења специфичног за *ESP32* платформу. Периодични прекид *FreeRTOS schedulera*, присутан по подразумеваном подешавању на оба језгра процесора, повремено прекида мерни прозор, уносећи додатне циклусе који немају везе са функцијом *fsm_transition()*. За разлику од *AVR* платформе, где је мерни механизам (*cli()/sei()* око бројача тајмера) довољан да елиминише све изворе недетерминизма будући да се извршава на *bare-metal* окружењу без оперативног система, на *ESP32* платформи *noInterrupts()/interrupts()* онемогућавају прекиде искључиво на језгру на коме се позивају, док друго језгро наставља да прима *FreeRTOS tick* прекид и може посредно утицати на тајминг мерења преко дељене меморијске магистрале. Ово ограничење је директно документовано од стране произвођача [ES2nd].

Детаљнији преглед максималних вредности по нивоу оптимизације показује да овај *jitter* није монотон са порастом оптимизације, како би се могло очекивати. За имплементације Директни *LUT FSM* (4840/1932/1930) и Планарни *HSM FSM* (8712/3883/2912) максимум опада на сваком следећем нивоу оптимизације. За имплементације *FSM* структурног низа (6782/1942/2895), *FSM* објеката стања (5832/2898/3862) и Угнеждени *HSM FSM* (11642/4858/7727), максимум на -O2 је већи него на -Os — код Угнеженог *HSM FSM*-а чак за 2869 циклуса. Овакво варирање вредности горе-доле без обрасца тачно је оно што би се очекивало да је прави узрок случајан. Ако бисмо претпоставили да *FreeRTOS* прекид утиче на мерење системски, све имплементације би на -O2 доследно ишле у истом правцу, уместо тога, пет иде на доле а три на горе, што показује да је у питању чиста случајност тајминга, не стваран ефекат оптимизације на *jitter*. Из овог разлога се за поређење перформанси на *ESP32* платформи као поуздана вредност користи минимум, а не просек или максимум. Максимум служи искључиво као показатељ горње границе *jittera* специфичног за платформу у датом појединачном покретању мерења, не као поновљива мера цене саме транзиције.

4.1.4 Упоредна анализа AVR и ESP32

Директно поређење броја циклуса између *AVR*-а (16 MHz) и *ESP32* (240 MHz) није смислено без превођења у апсолутно време, будући да се ради о различитим фреквенцијама и различитим скуповима инструкција. После нормализације (1 циклус *AVR* = 62,5 ns, 1 циклус *ESP32* = 4,17 ns), измерено убрзање на *ESP32* платформи креће се између 11,5x и 19,3x у зависности од имплементације, око теоријске вредности од 15x која произлази из самог односа фреквенција (240 MHz/16 MHz).

Одступање од ове теоријске вредности није случајно: имплементације са мало посла по транзицији (нпр. Модификована угнеждена *Switch-case FSM*, убрзање ~11,5x) остварују мање убрзање од теоријског, док имплементације са више посла по транзицији (нпр. Угнеждени *HSM FSM*, убрзање ~15x) остварују убрзање ближе теоријској вредности. Могуће објашњење лежи у архитектонској разлици у начину позивања функција — *Xtensa* архитектура по *default*-у користи механизам померајућих регистарских прозора (*eng. windowed registers*) за позиве функција (инструкције *CALL4/CALL8/CALL12*), који захтева додатну алокацију простора на стеку по позиву (16–32 додатна бајта, у зависности од дубине ротације) и може изазвати *window overflow exception* када се прозор пређе преко

границе расположивих регистара [CXnd], [ES3nd] - трошак који једноставан *call/ret* механизам AVR архитектуре нема.

Важно је нагласити да је овде реч о трошку синхроног позива функције (позив функције *fsm_transition()* из *measured_transition()*), не о *interrupt* латенцији описаној у одељку 3.3.1 у контексту *polling* методологије. Ова два механизма су независна: *interrupt* латенција настаје само при асинхроном прекиду тока програма спољашњим догађајем, док се трошак *windowed-register* позива јавља при сваком позиву функције, синхроно и предвидиво, без обзира на то да ли се мерење времена у датом тренутку ослања на *ISR* или на *polling* приступ. Код имплементација са мало посла по транзицији овај фиксни трошак чини већи удео укупног времена извршавања, умањујући остварено убрзање у односу на имплементације са више посла по позиву, код којих се исти фиксни трошак разлива преко веће базе времена извршавања.

Поредак имплементација по брзини на нивоу -O2 готово је исти на обе платформе. Директни *LUT FSM* и Угнеждена *Switch-case FSM* су најбржи на обе архитектуре, а Планарни и Угнеждени *HSM FSM* најспорији, опет на обе (Табела 5 и Табела 10). Једина разлика је што *FSM* структурног низа и *FSM* објеката стања замене места (на AVR-у је *FSM* објеката стања мало бржи, на ESP32-у обрнуто) — али та разлика је незнатна и износи свега 1–3 циклуса. Ово показује да то који приступ је бржи или спорији не зависи од тога на ком се процесору код извршава, већ од саме структуре приступа, да ли се транзиција решава једним директним читавањем из табеле, линеарном претрагом, или низом индиректних позива кроз хијерархију. Другим речима, ако је неки приступ бржи на AVR-у, са доста сигурности ће бити бржи и на ESP32-у — закључак који ће бити додатно проверен у одељцима 4.2 и 4.3, где се исти приступи мере на знатно већим аутоматима.

Огромна разлика у заузећу меморије између платформи (нпр. за Директни *LUT FSM* на -O2: 1978 В наспрам 271 537 В *Flash*-а, односно 302 В наспрам 21 472 В *RAM*-а, Табела 2 и Табела 7) не значи да је *FSM* код на ESP32-у стотинама пута већи — узрок је фиксни трошак улаза *Arduino-ESP32* развојног оквира, потврђен и квантификован мерењем *baseline sketch*-а у одељку 4.1.3 (Табела 6 и Табела 7). Када се тај фиксни трошак изолује, нето заузеће меморије по приступу постаје упоредиво по реду величине са AVR платформом, чиме поређење две платформе постаје методолошки ваљано и на нивоу заузећа меморије, не само на нивоу броја циклуса.

4.1.5 Разлагање трошка индиректних приступа

Поређење Директног *LUT FSM*-а и Индиректног *LUT FSM*-а (са индиректним позивом и провером граница) у одељку 4.1.2 показало је мерљиву разлику, али пошто тај приступ истовремено уводи два различита механизма, индиректан позив функције и експлицитну проверу граница пре приступа низу, сама по себи та разлика не приказује колики допринос потиче од сваког фактора појединачно.

Да би се ова два фактора раздвојила, уведена је трећа тачка поређења: имплементација са показивачима на функције (*Function Pointers*) [CO23], која садржи исти индиректан позив функције као и Индиректни *LUT FSM*, али нема проверу граница. Мерење је спроведено на обе платформе, чиме је могуће проверити да ли исти образац раздвајања трошка важи независно од архитектуре.

Табела 11 приказује измерени број циклуса за све три имплементације, на обе платформе, на сва три нивоа оптимизације.

Ниво	Директни LUT FSM (AVR)	Function Pointers (AVR)	Индиректни LUT FSM (AVR)	Директни LUT FSM (ESP32)	Function Pointers (ESP32)	Индиректни LUT FSM (ESP32)
-O0	79	102	82	74	91	87
-Os	17	30	38	18	30	39
-O2	18	30	37	14	28	37

Табела 11 - поређење броја циклуса за раздвајање трошка индиректног позива од трошка провере граница, AVR наспрам ESP32

Аритметичким раздвајањем ове три вредности (разлика *Function Pointers* – Директни *LUT FSM* представља чист трошак индиректног позива функције; разлика Индиректни *LUT FSM* – *Function Pointers* представља чист трошак додатне провере граница) добија се расподела у табели испод:

Ниво	Трошак индирекције, AVR	Трошак провере граница, AVR	Трошак индирекције, ESP32	Трошак провере граница, ESP32
-O0	+23	–20	+17	–4
-Os	+13	+8	+12	+9
-O2	+12	+7	+14	+9

Табела 12 - Трошак индиректних позива функција

На нивоима -Os и -O2, образац је доследан на обе платформе: индиректан позив функције кошта 12–14 циклуса, а додатна провера граница још 7–9 циклуса — оба фактора носе позитиван, кумулативан трошак, готово идентичне апсолутне вредности независно од архитектуре (Табела 12).

На нивоу -O0, уочен је неочекивани трошак на AVR платформи (Индиректни *LUT FSM* бржи од *Function Pointers*, упркос томе што садржи и индиректан позив и проверу граница) понавља се и на ESP32-у, само знатно мање изражено (–4 наспрам –20 циклуса). Објашњење остаје исто на обе платформе: *Function Pointers handler*-и враћају *uint8_t* вредност коју позивалац уписује у *current_state*. За разлику од њих акционе процедуре Индиректног *LUT FSM*-а имају тип *void* и саме, унутар тела функције, уписују *current_state* као споредни ефекат. На неоптимизованом коду, припрема и пренос повратне вредности при сваком позиву и повратку из функције носи мерљив трошак који делимично или потпуно надмашује цену додатне провере граница. Да је ефекат мање изражен на ESP32-у (–4 наспрам –20 циклуса) одражава разлику у механизму позива функција између две архитектуре — *Xtensa windowed registers* механизам (одељак 4.1.4) већ носи знатно већи фиксни трошак по позиву него AVR-ов *call/ret*, чиме додатни трошак преноса повратне вредности постаје пропорционално мањи удео укупне цене позива на ESP32-у него на AVR-у.

Овај налаз потврђује исти образац и на другој архитектури: на -O0 доминирају механичке разлике у припреми повратне вредности, а тек оптимизација уклања тај шум и открива праве, алгоритамске разлике међу приступима.

4.2 Резултати на проширеној (MQTT) машини стања

4.2.1 AVR платформа

Иницијално мерење циклуса на *MQTT* аутомату открило је неочекивано велика одступања за Директни *LUT FSM* (-Os, -O2), *FSM* објеката стања (-O2) и Показиваче на функције (-O2) — вишеструко (8–14х) веће просечне вредности него што теоријска оцена сложености предвиђа.

Начин имплементације	Ниво	Пре исправке	После исправке	Смањење
Директни LUT FSM	-Os	237	16	14.8х
Директни LUT FSM	-O2	243	21	11.6х
FSM објеката стања	-O2	250	28	8.9х
Показивачи на функције	-O2	253	34	7.4х

Табела 13 – одступање у броју циклуса за *MQTT*

Да би се утврдио узрок, генерисан је дисасемблирани испис извршног фајла наредбом *avr-objdump -d firmware.elf*, у ком је пажња усмерена на инструкције непосредно унутар мереног прозора функције *measured_transition()* — тачно између позива *cli()* и *cycles_stop()*. Испис је показао да преводилац, при укљученој *Link-Time Optimization (-flto)*, позив *rand()* оставља ван мереног прозора, али операцију остатка при дељењу (% NUM_EVENTS) премешта унутар мереног прозора, непосредно после *cli()* инструкције. Конкретно, у испису се непосредно после почетка мерења циклуса појављује позив софтверске рутине *_divmodhi4*. Рутину коју *AVR toolchain* уводи јер *AVR* архитектура нема хардверску инструкцију за 16-битно дељење, за разлику од множења. Ова рутина интерно извршава петљу од седамнаест итерација, чиме уноси неколико десетина додатних циклуса у мерени резултат, потпуно неповезаних са ценом саме *FSM* транзиције.

Исправка је спроведена заменом директног позива *rand() % NUM_EVENTS* унутар мерене петље унапред припремљеним низом од хиљаду насумично генерисаних догађаја (*static uint8_t precomputed_events[1000];*), попуњеним у потпуности пре почетка мерене петље. Овим се сваки позив *rand()* и свако дељење дешавају потпуно ван граница мереног прозора, чиме преводилац нема шта да премести преко границе. Идентична техника је примењена у мерењима на елементарном аутомату, где је *run_benchmark()* од самог почетка користио унапред дефинисан, циклички низ познат у време компајлирања. После исправке, сви претходно утврђени обрасци понављају се доследно и на овом, знатно већем аутомату.

Табела 14 приказује измерене вредности на нивоу -O0.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	101	106	112
Угнеждена Switch-case FSM (break)	113	118	123
FSM структурног низа	1	129	253
Директни LUT FSM	83	83	83
FSM објеката стања	105	113	125
Планарни HSM FSM	134	142	154
Угнеждени HSM FSM	222	222	222
Индиректни LUT FSM (Santic)	84	85	87

Табела 14 – број процесорских циклуса по транзицији MQTT аутомат, AVR платформа, са нивоом оптимизације -O0

Табела 15 приказује измерене вредности на нивоу -Os.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	29	34	36
Угнеждена Switch-case FSM (break)	29	34	36
FSM структурног низа	1	106	251
Директни LUT FSM	16	16	16
FSM објеката стања	26	29	34
Планарни HSM FSM	37	40	45
Угнеждени HSM FSM	80	89	95
Индиректни LUT FSM (Santic)	35	36	38

Табела 15 – број процесорских циклуса по транзицији MQTT аутомат, AVR платформа, са нивоом оптимизације -Os

Табела 16 приказује измерене вредности на нивоу -O2.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Максималан број циклуса
Модификована угнеждена Switch-case FSM (return)	28	33	36
Угнеждена Switch-case FSM (break)	28	33	36
FSM структурног низа	2	106	252
Директни LUT FSM	21	21	21
FSM објеката стања	25	28	33
Планарни HSM FSM	37	40	45
Угнеждени HSM FSM	79	89	94
Индиректни LUT FSM (Santic)	37	40	42

Табела 16 – број процесорских циклуса по транзицији, MQTT аутомат, AVR платформа, са нивоом оптимизације -O2

Разлика између *return* и *break* стилова писања угнеженог *switch-a* нестаје под оптимизацијом (идентичних 34/33 циклуса на -Os/-O2 за оба стила –Табела 15, Табела 16), потврђујући да је елиминација мртвог кода општа особина преводиоца, независна од величине аутомата. Директни *LUT FSM* задржава константан број циклуса на сваком нивоу (78/16/21), потврђујући $O(1)$ сложеност и на табели од 144 уноса. Однос између Планарног и Угнеженог *HSM FSM-a* (1.56–2.23x) остаје у распону утврђеном на елементарном аутомату, потврђујући да трошак хијерархијске пропагације нагоре не зависи од укупне величине аутомата, већ искључиво од броја нивоа кроз које догађај пропагира.

Међутим, унета исправка са собом носи одређену цену. Коришћење статички алоцираног низа за унапред генерисане догађаје значајно повећава заузеће *SRAM* меморије у односу на верзију без исправке, што на *AVR* платформи, због ограниченог укупног капацитета радне меморије, представља нетривијалну последицу, која ће бити детаљно објашњена у наставку.

4.2.2 ESP32 платформа

Пре мерења, спроведена је контролна провера да ли исти *-fno* утицај откривен на *AVR* платформи (одељак [X]) постоји и на *ESP32 (Xtensa)* архитектури. За два pattern-a (*Nested Switch — return* и *break* варијанте) измерен је просечан број циклуса и са унапред генерисаним низом догађаја, и са директним позивом *rand() % NUM_EVENTS* унутар мерене петље.

Ниво оптимизације	СА низом	БЕЗ низа	Разлика
return -O0	132	133	занемарљиво (± 1 , шум)
return -Os	60	60	идентично
return -O2	59	59	идентично
break -O0	157	157	идентично
break -Os	60	60	идентично

Табела 17 - контролно поређење са/без унапред генерисаног низа догађаја ESP32/MQT

Резултати показују занемарљиву разлику (испод 1 циклуса, у оквиру нормалног мерног шума) на сва три нивоа оптимизације, за оба стила писања кода, за разлику од *AVR* платформе, где је идентична измена узроковала разлику од 7 до 15 пута. Ово потврђује да *Xtensa* архитектура, захваљујући хардверској подршци за операцију дељења, не пати од истог начина премештања инструкција, чиме је *precomputed_events* низ на *ESP32* платформи задржан искључиво ради методолошке доследности мерења између две платформе, не из функционалне неопходности. Уклањањем овог низа радна меморија смањује се за тачно 1000 бајтова по имплементацији, што за свих једанаест тестираних имплементација доноси уштеду од укупно 11000 бајтова — иако, за разлику од *AVR*-а, *ESP32* (~320 KB SRAM-а) овај трошак не доводи у питање стабилност рада система, реч је и даље о непотребном трошку без функционалног оправдања на овој архитектури.

Ради упоредивости са *AVR* резултатима, *Flash* заузеће представљено је као нето вредност, добијена одузимањем *baseline* мерења празног *sketch*-а (Табела 6) од сваког сировог резултата — иста методологија примењена на елементарном аутомату.

Начин имплементације	Flash нето -O0 (B)	Flash нето -Os (B)	Flash нето -O2 (B)
Директни LUT FSM	2504	1864	1936
Модификована угнеждена Switch-case FSM (return)	3196	2160	2348
Угнеждена Switch-case FSM (break)	3728	2160	2348
FSM структурног низа	2896	2180	2228
FSM објеката стања	3528	2336	2384
Планарни HSM FSM	3684	2424	2492
Угнеждени HSM FSM	4756	2704	2824
Индиректни LUT FSM (Santic)	5188	4000	4060

Табела 18 – нето заузеће *Flash* меморије по приступу на *MQTT* аутомату, *ESP32* платформа

Након изолације фиксног трошка развојног оквира, апсолутне вредности постају упоредиве по реду величине са *AVR* резултатима (нпр. Директни *LUT FSM*: 1936 В нето на *ESP32* наспрам 2552 В на *AVR* — исти ред величине, за разлику од сирове разлике од стотина хиљада бајтова). Индиректни *LUT FSM (Santic)* издваја се као апсолутно највећа имплементација на оба нивоа оптимизације (4000–5188 В) — очекивано, с обзиром да укључује 144 засебне *void* процедуре плус табелу од 144 показивача на функције, знатно већи код-отисак него било који други приступ.

Табела 19 приказује измерене вредности на нивоу -O0.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Макс. број циклуса
Модификована угнеждена Switch-case FSM (return)	85	133	7739

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Макс. број циклуса
Угнеждена Switch-case FSM (break)	97	157	8705
FSM структурног низа	692	2578	7539
Директни LUT FSM	74	81	2925
FSM објеката стања	102	153	4878
Планарни HSM FSM	130	183	7767
Угнеждени HSM FSM	326	397	15535
Индиректни LUT FSM (Santic)	83	161	3889

Табела 19 – број процесорских циклуса по транзицији MQTT, ESP32, са нивоом оптимизације -O0

Табела 20 приказује измерене вредности на нивоу -Os.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Макс. број циклуса
Модификована угнеждена Switch-case FSM (return)	23	60	3872
Угнеждена Switch-case FSM (break)	23	60	3872
FSM структурног низа	207	801	5597
Директни LUT FSM	18	24	1930
FSM објеката стања	29	53	3874
Планарни HSM FSM	40	66	4826
Угнеждени HSM FSM	80	117	6767
Индиректни LUT FSM (Santic)	36	103	4827

Табела 20 – број процесорских циклуса по транзицији MQTT аутомат, ESP32 платформа на нивоу оптимизације -Os

Табела 21 приказује измерене вредности на нивоу -O2.

Начин имплементације	Минималан број циклуса	Просечан број циклуса	Макс. број циклуса
Модификована угнеждена Switch-case FSM (return)	21	59	4828
Угнеждена Switch-case FSM (break)	21	59	4828
FSM структурног низа	217	870	5676
Директни LUT FSM	21	27	1934
FSM објеката стања	36	59	3869
Планарни HSM FSM	40	65	4827
Угнеждени HSM FSM	81	123	7750
Индиректни LUT FSM (Santic)	37	102	3877

Табела 21 – број процесорских циклуса по транзицији, MQTT аутомат ESP32, са нивоом оптимизације -O2

Return наспрам *break* стилска разлика потпуно нестаје под оптимизацијом, четврта независна потврда истог обрасца у овом раду. На нивоу -O0, *break* варијанта захтева мерљиво више циклуса од *return* варијанте (157 наспрам 133, разлика од 18%) —Табела 19. На -Os и -O2, обе варијанте дају идентичне резултате до бајта — исти број циклуса (60, затим 59), исто заузеће *Flash* меморије. Ово представља четврту независну потврду исте појаве елиминације мртвог кода (*AVR/Access Control*, *AVR/MQTT*, *ESP32/Access Control*, сада и *ESP32/MQTT*), што га чврсто утемељује као општу, архитектонски независну особину оптимизационих пролаза.

FSM структурног низа показује драматично, доследно најслабије перформансе — најизраженији $O(n)$ налаз у овом раду. За разлику од свих осталих приступа, чији се просечан број циклуса на -O2 креће у распону 27–123, *FSM* структурног низа захтева 870 циклуса у просеку — 32 пута више од Директног *LUT FSM*-а (27 циклуса) на истом аутомату. Овај однос значајно превазилази раније измерене разлике на мањем аутомату (реда величине 2–3х, не 30х), очекивано с обзиром на величину табеле транзиција (144 уноса наспрам свега 5). Пошто се тренутно стање аутомата у овом *benchmark*-у мења кроз стварну путању изазвану насумичним догађајима, измерена минимална/просечна вредност зависи од тога које је позиције у табели тренутна путања посетила у датом покретању, што апсолутне бројеве чини мање директно упоредивим мерење-по-мерање са мањом машином стања, али не умањује валидност закључка о правцу и реду величине разлике.

Хијерархијска пропагација нагоре остаје доследна, умерено скупа. Планарни *HSM FSM* наспрам Угнежденог *HSM FSM* показује однос од 1.77–2.17х на сва три нивоа

оптимизације (Табела 19, Табела 20, Табела 21), у складу са распоном већ утврђеним на *AVR* платформи и на мањем аутомату. Независна потврда да је трошак једног нивоа хијерархијске пропагације нагоре стабилна, предвидива карактеристика механизма.

Индиректни *LUT FSM* наспрам Директног *LUT FSM*-а показује стабилан трошак индиректног позива и провере граница — разлика износи 75–80 циклуса доследно на сва три нивоа оптимизације (наспрам *AVR*-ове *Access Control* мере од 12–21 циклус). Већи апсолутни трошак овде потиче делом од веће табеле, делом од тога што *MQTT benchmark*, за разлику од *AVR* верзије, не користи унапред генерисан низ догађаја (пошто је контролним мерењем на почетку овог одељка потврђено да *ESP32* нема превеликог увећања броја циклуса приликом транзиције током инструкције премештања дељења), што уноси изванредан, стабилан додатни трошак генерисања насумичног броја подједнако код свих приступа.

Заузеће меморије остаје доминантно одређено оквиром, не *FSM* кодом — *Flash* заузеће креће се у уском опсегу кроз свих осам приступа (разлика од свега 6.6%), док *RAM* заузеће остаје практично константно (разлика од само 8 бајтова), у складу са раније утврђеним налазом на елементарном аутомату.

Резултати мерења на *MQTT* аутомату независно потврђују сваки од централних налаза утврђених на мањем, *Access Control* аутомату — стилске разлике у писању кода нестају под оптимизацијом преводиоца, трошак хијерархијске пропагације нагоре остаје стабилан и предвидив, а трошак индиректних позива функција преко показивача остаје доследно мерљив, независно од платформе и величине аутомата.

Специфично за *ESP32/MQTT* комбинацију, скалирање линеарне претраге на знатно већу табелу транзиција открива драматичнији раст трошка него што је био видљив на мањем аутомату, чинећи разлику између $O(1)$ и $O(n)$ приступа диспечовања знатно упечатљивијом на аутомату реалистичније величине.

Максималне измерене вредности у табелама (Табела 19, Табела 20, Табела 21), показују исти образац нестабилности већ документован на елементарном аутомату (одељак 4.1.3) — повремено достижу и по неколико хиљада циклуса (нпр. Угнеждени *HSM FSM* на -O0: минимум 326, максимум 15 535), услед истог *FreeRTOS scheduler* прекида на другом језгру процесора који *noInterrupts()/interrupts()* не могу да онемогуће. Ова нестабилност остаје немонотона са порастом оптимизације и на *MQTT* аутомату. Нпр. *FSM* структурног низа на -O2 показује виши максимум (5676) него на -Os (5597), упркос нижем просеку — потврђујући да је реч о случајном тајмингу, а не систематском ефекту оптимизације.

Из истог разлога наведеног у одељку 4.1.3, за поређење перформанси на *ESP32* платформи и на *MQTT* аутомату као поуздана вредност користи се минимум, а не просек или максимум. Максимум остаје искључиво индикатор горње границе *jittera* специфичног за платформу у датом покретању, не поновљива мера цене транзиције.

4.2.3 Упоредна анализа *AVR* и *ESP32*

Нормализацијом мереног броја циклуса у апсолутно време (1 циклус *AVR* = 62,5 ns, 1 циклус *ESP32* = 4,17 ns), измерено убрзање на *ESP32* платформи за *MQTT* аутомат креће се, за седам од осам приступа, у распону 10,4–19,98х на нивоу -O2 — готово идентичан опсег утврђеном

на елементарном аутомату (11,5–19,3х, одељак 4.1.4). Ово представља независну потврду да однос перформанси између две платформе не зависи од величине тестираног аутомата, чиме се закључак из одељка 4.1.4 генерализује изван граница минималног тест-примера, у складу са најавом датом у том одељку.

FSM структурног низа изостављен је из овог опсега: минимална измерена вредност на *AVR* платформи (свега 2 циклуса на -O2) представља статистичка аномалија методологије мерења описане у одељку 4.2.2, где се тренутно стање аутомата током *benchmark*-а мења кроз насумичну путању — минимална вредност у датом покретању зависи од тога да ли је путања у том тренутку погодила позицију близу почетка табеле транзиција, не од стварне, типичне цене претраге. Ова специфичност чини директно поређење минималних вредности за овај приступ методолошки непоузданим, за разлику од свих осталих ($O(1)$ и *HSM*) приступа, где минимум доследно одражава фиксну, поновљиву цену транзиције.

Поредак имплементација по брзини на нивоу -O2 остаје доследан на обе платформе и на *MQTT* скали. Директни *LUT FSM* и Индиректни *LUT FSM* деле најбоље место (14,99х на обе), Угнеждена *Switch-case FSM* показује највеће убрзање (19,98х) захваљујући најмањем фиксном раду по транзицији, док Угнеждени *HSM FSM* остаје међу спорије убрзаним приступима (14,62х) услед два узастопна индиректна позива кроз показивач на функцију при хијерархијској пропагацији нагоре. Исти механизам *windowed registers* трошка описан у одељку 4.1.4 [CXnd], [ES3nd], сада потврђен и на аутомату са четири нивоа хијерархије уместо једног.

Поређење нето заузећа *Flash* меморије (одељак 4.2.2) показује да апсолутне вредности остају упоредиве по реду величине између платформи и на *MQTT* скали. Директни *LUT FSM* захтева 1936 В нето на *ESP32* (-O2) наспрам 2552 В на *AVR*-у, исти ред величине, за разлику од сирове разлике реда стотина хиљада бајтова пре одузимања *baseline* вредности.

Релативни однос перформанси између имплементационих приступа: који је бржи, који захтева мање меморије, колики је трошак хијерархијске пропагације или индиректног позива функције остаје доследан кроз две архитектонски различите платформе (8-битни *AVR* и 32-битни *Xtensa*) и кроз два аутомата различите скале (пет наспрам сто четрдесет четири транзиција). Ово представља јаку, двоструко независну потврду да закључци овог рада нису специфичност једне платформе нити једног, намерно малог тест-примера, већ одражавају суштинска, структурна својства самих имплементационих приступа.

4.3 Резултати на комплексној машини стања

4.3.1 AVR платформа

4.3.2 ESP32 платформа

4.3.3 Упоредна анализа AVR и ESP32

4.4 Поређење скалирања кроз три нивоа машине стања

(елементарна → проширена → комплексна, кључни закључак целог рада)

4.5 Закључна дискусија резултата

Литература

[PA06] Adamczyk, P. *The Anthology of the Finite State Machine Design Patterns*, 2006.

[CO23] Carlgren, J., Oskarsson, P. W. *State Machine Model-To-Code Transformation In C*. UPTec F 23044, Uppsala University, 2023.

[YA98] Yacoub, S., Ammar, H. *A Pattern Language of Statecharts*. 1998.

[MV03] Moreno, A., Valduvieto, J. *dFSM: Finite State Machines for Embedded Systems*. Real-Time Linux Workshop (RTLWS), 2003.

[KJH23] Kadam, S., Jogalekar, S., Hembade, S. *Model a Finite State Machine as a Construct in Computer Programming*. Journal of Data Acquisition and Processing, Vol. 38 (1), 2023.

[DH87] Harel, D. *Statecharts: A Visual Formalism for Complex Systems*. Science of Computer Programming, Vol. 8, str. 231–274, 1987.

[JSnd] Santic, J. *Writing Efficient State Machines in C*. Dostupno na: <http://johnsantic.com/comp/state.html> (pristupljeno 2026).

[AK17] Kumar, A. *How to implement finite state machine in C*. Articleworld, 13. avgust 2017. Dostupno na: <https://articleworld.com/state-machine-using-c/>

[MS02] Samek, M. *Practical Statecharts in C/C++*. CMP Books, 2002.

[SS19] Sunitha, E. V., Samuel, P. *Automatic Code Generation From UML State Chart Diagrams*. IEEE Access, 7:8591–8608, 2019.

[ESnd] Espressif Systems. *ESP-IDF Programming Guide — Memory Types (ESP32)*. Dostupno na: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/memory-types.html> (pristupljeno 2026).

[ES2nd] Espressif Systems. *ESP-IDF Programming Guide — ESP-IDF FreeRTOS SMP Changes*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/freertos-smp.html> (pristupljeno 2026).

[AMnd] Atmel / Microchip. *ATmega328P Datasheet Complete*. Dostupno na: https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328_P%20AVR%20MCU%20with%20picoPower%20Technology%20Data%20Sheet%2040001984A.pdf (pristupljeno 2026).

[GNUnd] GNU Project. *Using the GNU Compiler Collection — Options That Control Optimization*. Одељак 3.10. Доступно на: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> (приступљено 2026).

[MCnd] Microchip Technology. *AVR-GCC Optimization Guide* /Nadji nadja link gde si ga nasla

[CXnd] Cadence Design Systems, Inc. *Xtensa® Instruction Set Architecture (ISA) Summary for all Xtensa LX Processors*. Доступно на: https://www.cadence.com/content/dam/cadence-www/global/en_US/documents/tools/silicon-solutions/compute-ip/isa-summary.pdf (приступљено 2026).

[ES3nd] Espressif Systems. *Overview of Xtensa ISA*. Доступно на: https://dl.espressif.com/github_assets/espressif/xtensa-isa-doc/releases/download/latest/Xtensa.pdf (приступљено 2026).

[TAB17] Tappler, M., Aichernig, B.K., Bloem, R. *Model-Based Testing IoT Communication via Active Automata Learning*. У: 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST 2017), стр. 276–287. IEEE Computer Society, 2017.

[AUTOnd] Radboud University, iCIS. *Automata Wiki — Model details*. Доступно на: <https://automata.cs.ru.nl/Table> (приступљено 2026).